



FLORIDA STATE UNIVERSITY
DEPARTMENT OF SCIENTIFIC COMPUTING

SUMMER 2024

Written Preliminary Examination Report

Submitted By:
Mani Tyagi

Student ID :
200973108
mt22t

May 23, 2024

Contents

1	Fast Fourier Transfromation [Dr. Meyer-Baese]	3
1.1	Part 1	3
1.2	Part 2	3
2	Linear Algebra [Dr. Quaife]	4
2.1	Part (a)	4
2.2	Part (b)	4
2.3	Part (c)	5
2.4	Part (d)	7
2.5	References	7
3	Ordinary Differential Equation [Dr. Quaife]	8
3.1	Part (a)	8
3.2	Part (b)	9
3.3	Part (c)	12
3.4	Part (d)	12
3.5	Part (e)	13
3.6	References	14
4	Numerical Integration [Dr. Huang]	15
4.1	Part (a)	15
4.2	Part (b)	16
4.3	Part (c)	17
4.4	References	18
5	Approximation and Differentiation [Dr. Huang]	19
5.1	Part (a)	19
5.2	Part (b)	19
5.3	Part (c)	20
5.4	Part (d)	21
5.5	Part (e)	22
5.6	References	22
6	Data Structures [Dr. Erlebacher]	23
7	Maximum Likelihood Estimation [Dr. Chipilski]	24
7.1	Part (a)	24
7.2	Part (b)	24
7.3	Part (c)	25
7.4	Part (d)	26
7.5	Part (e)	26
7.6	References	27
8	Parallel Programming [Dr. Zavala]	28
8.1	OpenMP	28
8.2	MPI	30
8.3	References	30
9	Statistics [Dr. Beerli]	31
9.1	Part 1	31

9.2	Part 2	32
9.3	Part 3	32
9.4	Part 4	34
9.5	Part 5	36
9.6	References	36
10	Numerical Integration[Dr. Sachin]	37
10.1	Part (a)	37
10.2	Part (b)	38
10.3	Part (c)	39
10.4	Appendix: Discontinuity of $f'_n(x)$ at $x = 0$	41
10.5	References	42
11	Parallel Programming[Dr. Wang]	43
11.1	Serial Code	43
11.2	Parallel Code and Load balancing	45
11.3	Scalability of Code	46
11.4	References	47

1 Fast Fourier Transformation[Dr. Meyer-Baese]

1.1 Part 1

n_1	n_2			
	0	1	2	3
0	x[0]	x[3]	x[6]	x[9]
1	x[4]	x[7]	x[10]	x[1]
2	x[8]	x[11]	x[2]	x[5]

k_1	k_2			
	0	1	2	3
0	X[0]	X[9]	X[6]	X[3]
1	X[4]	X[1]	X[10]	X[7]
2	X[8]	X[5]	X[2]	X[11]

Figure 1: Tables for index map.

1.2 Part 2

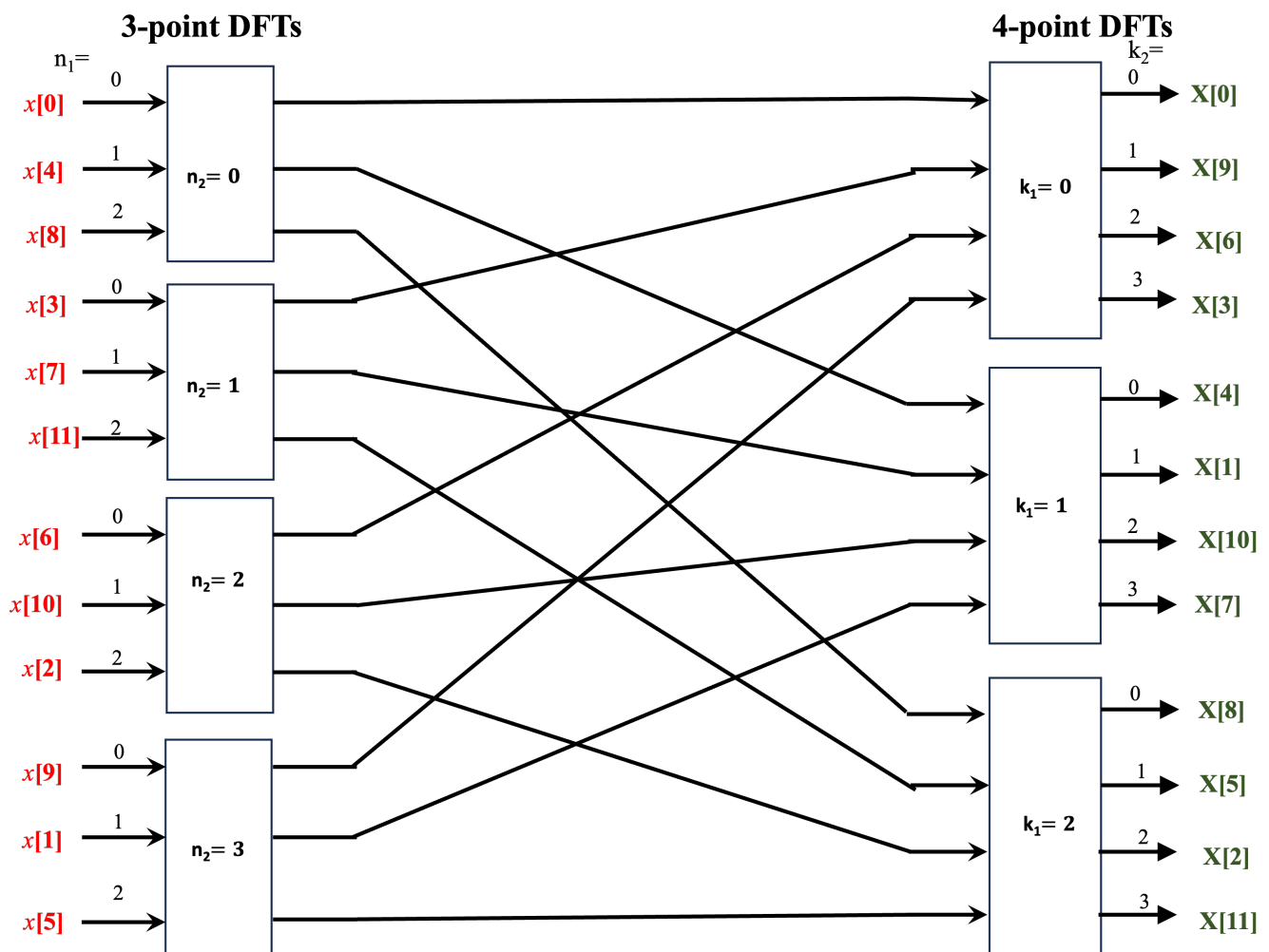


Figure 2: SFG for given FFT.

For completing this question, I have taken help from the examples given in [4].

2 Linear Algebra[Dr. Quaife]

The given minimization problem is

$$\min_{\mathbf{w} \in R^L} \left\{ \sum_{i=1}^N \left(y_i - \sum_{\ell=1}^L w_{\ell} \phi_{\ell}(x_i) \right)^2 \right\}$$

In modified form it can be written as

$$\min_{\mathbf{w} \in R^L} \|\mathbf{y} - X\mathbf{w}\|^2$$

2.1 Part (a)

The matrix X can be constructed using the given basis functions $\phi_j(x)$ with $(j = 1, 2, 3, 4, \dots, L)$ and the data points (x_i, y_i) . Each row of X corresponds to a data point (x_i, y_i) , and each column corresponds to a basis function $\phi_j(x)$. The element X_{ij} represents the value of the j -th basis function evaluated at the i -th data point x_i .

Therefore, the matrix X can be written as:

$$X = \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_L(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_L(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_L(x_N) \end{bmatrix}$$

Each row represents a data point, and each column represents a basis function evaluated at all data points. The given basis function is

$$\phi_i(x) = \exp(-1000(x - c_i)^2) \quad \text{where } i = 1, \dots, 100$$

2.2 Part (b)

A code that takes x and y values along with c values is written in attached python notebook - Q2.ipynb

```

1  def solve_least_squares(x, y, c_values, lambda_val):
2      """-----
3      Function: Solve the regularized least squares problem.
4
5      Parameters:
6          x : The input values with shape (N,).
7          y : The target values with shape (N,).
8          c_values : The values of basis function centers with shape (L,).
9          lambda_val : The regularization parameter.
10
11     Returns:
12         - error and weight norm.
13     -----"""
14     N = len(x) # Number of samples
15     L = len(c_values) # Number of basis functions
16     I = np.eye(L) # Identity matrix of shape (L, L)
17
18     # Create the design matrix X

```

```

19 X = np.zeros((N, L))
20 X = create_X(x, c_values)
21
22 # Solve the regularized least squares problem
23 w = np.linalg.solve(X.T @ X + lambda_val * I, X.T @ y)
24
25 # Evaluate error and coefficient norm
26 error = np.linalg.norm(y - X @ w) ** 2
27 weight_norm = np.linalg.norm(w) ** 2
28
29 return error, weight_norm

1 # Testing the function
2 N = 10
3 x = np.linspace(0, 1, N)
4 y = np.sin(2 * np.pi * x) + 0.1*np.sin(20*np.pi*x)
5 # Define basis functions
6 c_values = np.linspace(0, 1, 100)
7 lambda_val = 0.01
8 error, weight_norm = solve_least_squares(x, y, c_values, lambda_val)
9 print("Error:", error)
10 print("Weight Norm:", weight_norm)

```

Results:

```

-----
Error: 3.496512804482613e-05
Weight Norm: 1.376303252276021
-----

```

2.3 Part (c)

The given regularized optimization problem is

$$\min_{\mathbf{w} \in \mathbb{R}^L} (\|\mathbf{y} - X\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2)$$

The first term in this equation is the objective function from the ordinary least square(OLS) regression [1]. The second term is a regularization term, not present in OLS, which penalizes large w values. The normal equations for this optimization are

$$(X^T X + \lambda I)w = X^T y$$

where I is the $L \times L$ identity matrix.

λ	$\ \mathbf{y} - X\mathbf{w}\ ^2$	$\ \mathbf{w}\ ^2$
0	0.000000	29106.743685
10^{-8}	0.000000	40.597778
10^{-6}	0.000004	6.777142
0.0001	0.000052	2.417206
0.01	0.000703	1.844352
1	0.080763	1.611000
100	29.642011	0.088426

Table 1: Results of Regularized Least Squares

If the assumptions of ordinary least squares (OLS) regression are met, the solution $w = (X^T X)^{-1} X^T y$ with $\lambda = 0$ is both an unbiased estimator and the minimum-variance linear unbiased estimator. Introducing the term λI leads to a biased solution, but it also reduces variance. Hence, larger values of λ result in lower variance. Thus, adjusting λ manages the trade-off between bias and variance. This behaviour is obtained in the solution of the given problem as well. For small λ norm of error is small. But for larger values of regularization parameter λ , the error norm increases.

The results obtained can be summarized as follows:

- **No Regularization ($\lambda = 0$):**
 - **Error:** Very low (perfect fit)
 - **Norm of Coefficients:** Very high (potential over fitting)
- **Small λ Values (e.g., $1e-08$, $1e-06$):**
 - **Error:** Close to zero
 - **Norm of Coefficients:** Significantly reduced
- **Moderate λ Values (e.g., 0.0001 , 0.01):**
 - **Error:** Slightly increases
 - **Norm of Coefficients:** Further decreased
- **Large λ Values (e.g., 1 , 100):**
 - **Error:** Increases significantly
 - **Norm of Coefficients:** Very low

Conclusion:

- The choice of λ is critical to achieving a balance between fitting the data well and maintaining a simple model to prevent over-fitting.
- Small to moderate values of λ often provide a good trade-off, whereas very large values can lead to under-fitting.

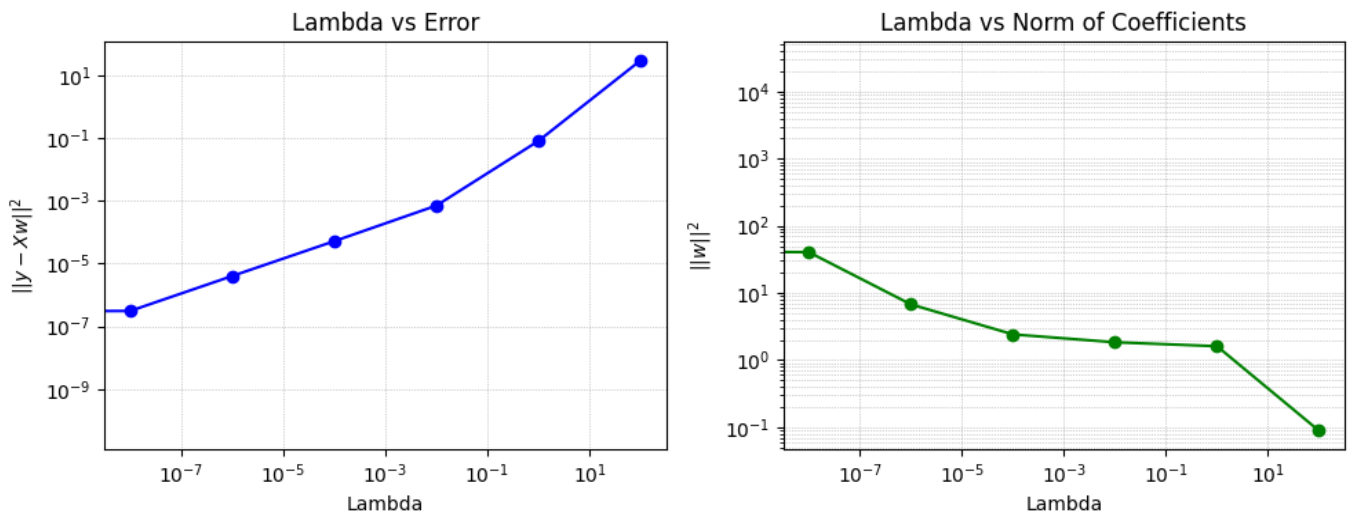


Figure 3: Plot demonstrating changes in error and norm of coefficient with variations in λ .

2.4 Part (d)

λ	$\ y - Xw\ ^2$	$\ w\ ^2$	$\ w\ _{GMRES}^2$	GMRES Iterations
0	0.000000	29106.743685	12.264043	100000
10^{-8}	0.000000	40.597778	12.078175	100000
10^{-6}	0.000004	6.777142	6.749112	100000
0.0001	0.000052	2.417206	2.417206	80557
0.01	0.000703	1.844352	1.844352	945
1	0.080763	1.611000	1.611000	29
100	29.642011	0.088426	0.088426	9

Table 2: Results of Regularized Least Squares with GMRES. `max_iter` = 100000

Discussion: For small values of regularization parameter λ , GMRES method fails to converge even for very large number of iterations. And as value of λ is increased, the iteration count decreases quite rapidly, suggesting a good convergence.

Explanation: The Gmres method does not reaches convergence for small λ values. This is because for small λ values $X^T X + \lambda I$ is ill conditioned. And these unpreconditioned iterative methods struggle to converge in such cases [5]. The condition number for different values of λ is shown in Table-3 and explains the convergence failure for small λ .

λ	$\text{Cond}(X^T X + \lambda I)$
0	10385676005557310.000000
10^{-8}	3064975467.217900
10^{-6}	30649764.951245
0.0001	306498.640253
0.01	3065.976403
1	31.649764
100	1.306498

Table 3: Condition number of $X^T X + \lambda I$ for different values of λ .

Note: When I used GMRES function on the first day of exam, I was not able to retrieve number of iterations. It was giving exit codes of 0 (meaning convergence of result) and `max_iter` (meaning failure to converge). On the last day, I revisited this issue and used ChatGpt ([conversation](#)) to help me.

2.5 References

1. Scipy documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.gmres.html>
2. [Saunders - Least Squares](#)

3 Ordinary Differential Equation[Dr. Quaife]

The given system of initial value problem:

$$\frac{dL}{dt} = f_1(L, X, Y) = -4\alpha X - 4800L, \quad (1)$$

$$\frac{dX}{dt} = f_2(L, X, Y) = -5L(Y - 0.01) - 330X, \quad (2)$$

$$\frac{dY}{dt} = f_3(L, X, Y) = 5LX - 330(Y - 0.1) \quad (3)$$

3.1 Part (a)

Code for testing the solver with $\alpha = 0.1$

```

1 # Define the system of differential equations
2 def system(t, variables, alpha):
3     L, X, Y = variables
4     dL_dt = -4 * alpha * X - 4800 * L
5     dX_dt = -5 * L * (Y - 0.01) - 330 * X
6     dY_dt = 5 * L * X - 330 * (Y - 0.1)
7     return [dL_dt, dX_dt, dY_dt]
8
9 # initial conditions
10 initial_conditions = [0.01, 0.01, 0.1]
11
12 # alpha
13 alpha = 0.1
14
15 # time for which to solve the system
16 t_span = [0, 1]
17
18 # Solve the system of IVPs using solve_ivp
19 solution = solve_ivp(system, t_span, initial_conditions, args=(alpha,),
20                       method='RK45')
21
22 # Printing the solution
23 print("Time Points (t):\n", solution.t)
24 print("\nVariables (L, X, Y):")
25 for i, variable in enumerate(['\nL: ', '\nX: ', '\nY: ']):
26     print(f"{variable} {solution.y[i]}")

```

RESULTS:

Time Points (t):

[0. 0.000196 0.000392 ... 0.998839 0.999602 1.]

Dependent Variables (L, X, Y):

L: [0.01 0.003902 0.001523 ... 0.000001 0.000002 0.]

X: [0.01 0.009373 0.008785 ... 0. 0. 0.]

Y: [0.1 0.1 0.1 ... 0.1 0.1 0.1]

3.2 Part (b)

The IVP has three fixed points. If a solution reaches one of these fixed points, it stays there indefinitely.

First Fixed Point:

$$(L_1, X_1, Y_1) = (0, 0, 0.1)$$

Second and Third Fixed Points:

$$L_2 = 66\sqrt{\frac{\alpha}{880000} - 1}, \quad X_2 = -\frac{1200}{\alpha}L_2, \quad Y_2 = 0.1 - \frac{200}{11\alpha}L_2^2$$

$$L_3 = -66\sqrt{\frac{\alpha}{880000} - 1}, \quad X_3 = \frac{1200}{\alpha}L_3, \quad Y_3 = 0.1 - \frac{200}{11\alpha}L_3^2$$

For $\alpha = 1 \times 10^6$

Fixed points for alpha = 1000000.0:

Fixed Point 1: (0, 0, 0.1)

Fixed Point 2: (24.372115, -0.029247, 0.089200)

Fixed Point 3: (-24.372115, 0.029247, 0.089200)

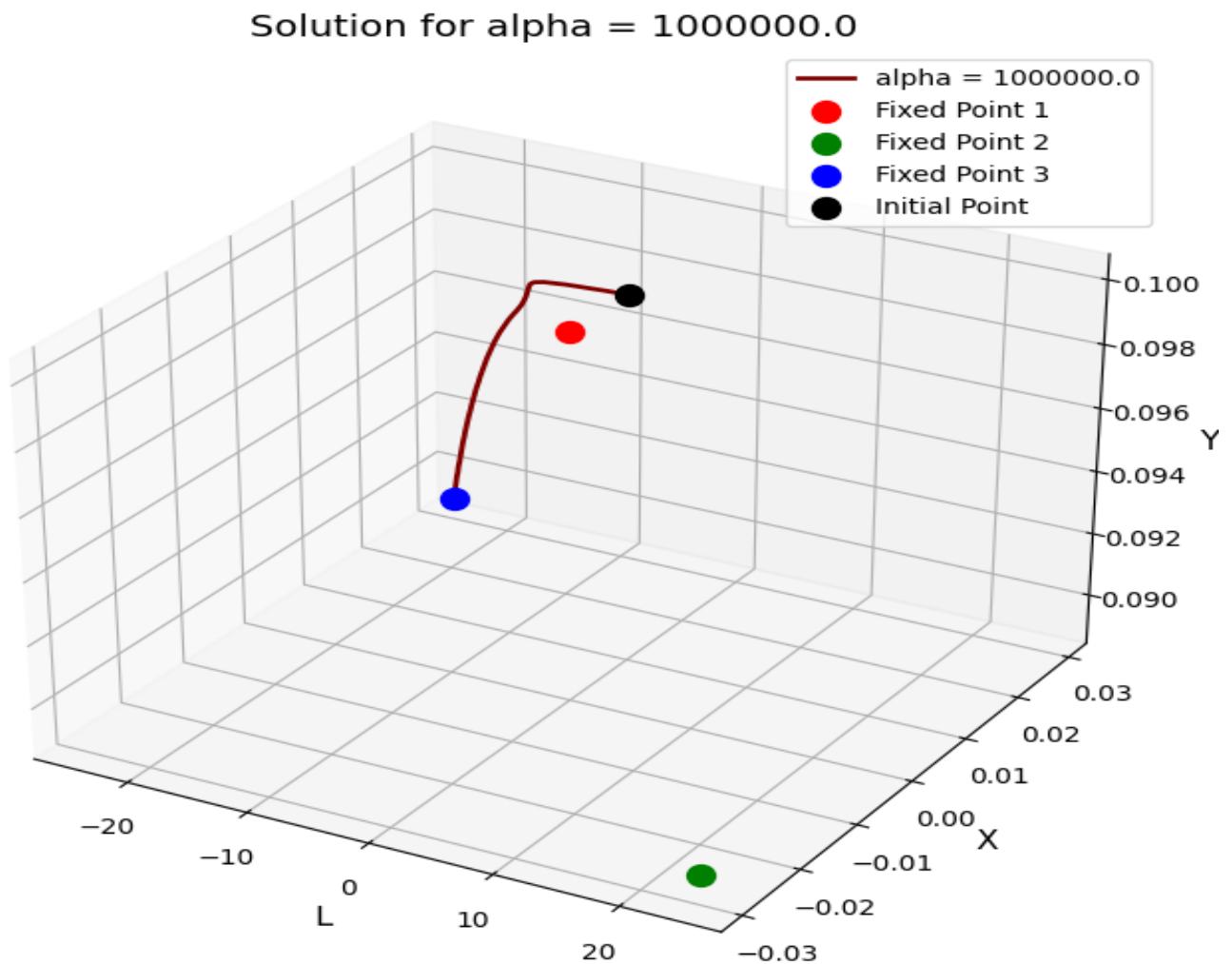


Figure 4: Solution for $\alpha = 1 \times 10^6$.

Behavior: For $\alpha = 1 \times 10^6$, the system quickly stabilizes and stays at fixed point (L_3, X_3, Y_3) . Afterwards, it remains there for the entire timespan.

For $\alpha = 3 \times 10^6$

Fixed points for alpha = 3000000.0:

Fixed Point 1: (0, 0, 0.1)

Fixed Point 2: (102.440226, -0.040976, 0.036400)

Fixed Point 3: (-102.440226, 0.040976, 0.036400)

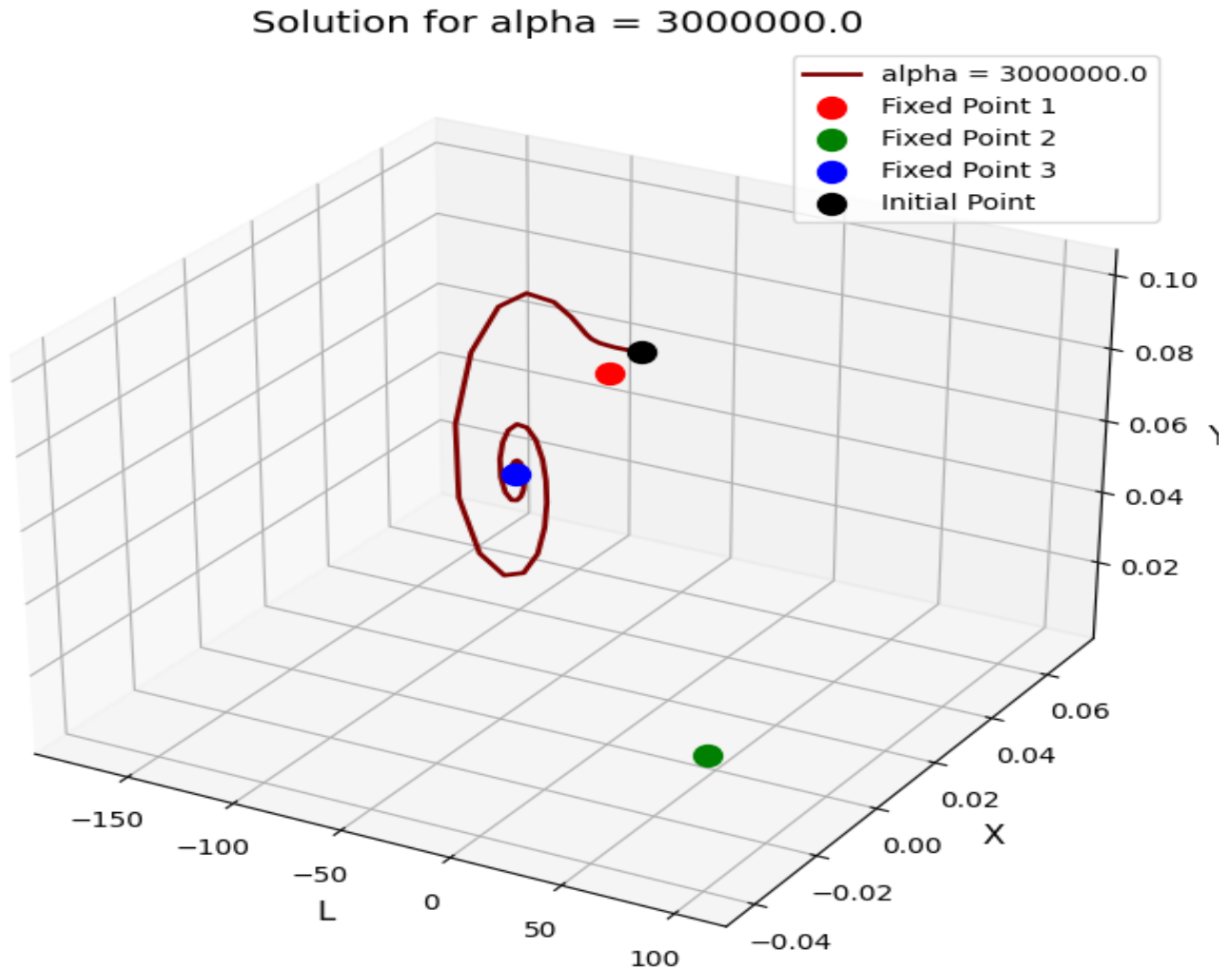


Figure 5: Solution for $\alpha = 3 \times 10^6$.

Behavior: For $\alpha = 3 \times 10^6$, the system takes some time to stabilize and then comes at fixed point (L_3, X_3, Y_3) . Afterwards, it remains there for the entire time span. The fixed points 2 and 3, in this case, are far away from the origin. Because of this the solution takes some time, oscillate and then reaches the fixed point 3.

For $\alpha = 2 \times 10^7$

Fixed points for alpha = 20000000.0:

Fixed Point 1: (0, 0, 0.1)

Fixed Point 2: (307.642650, -0.018459, 0.013960)

Fixed Point 3: (-307.642650, 0.018459, 0.013960)

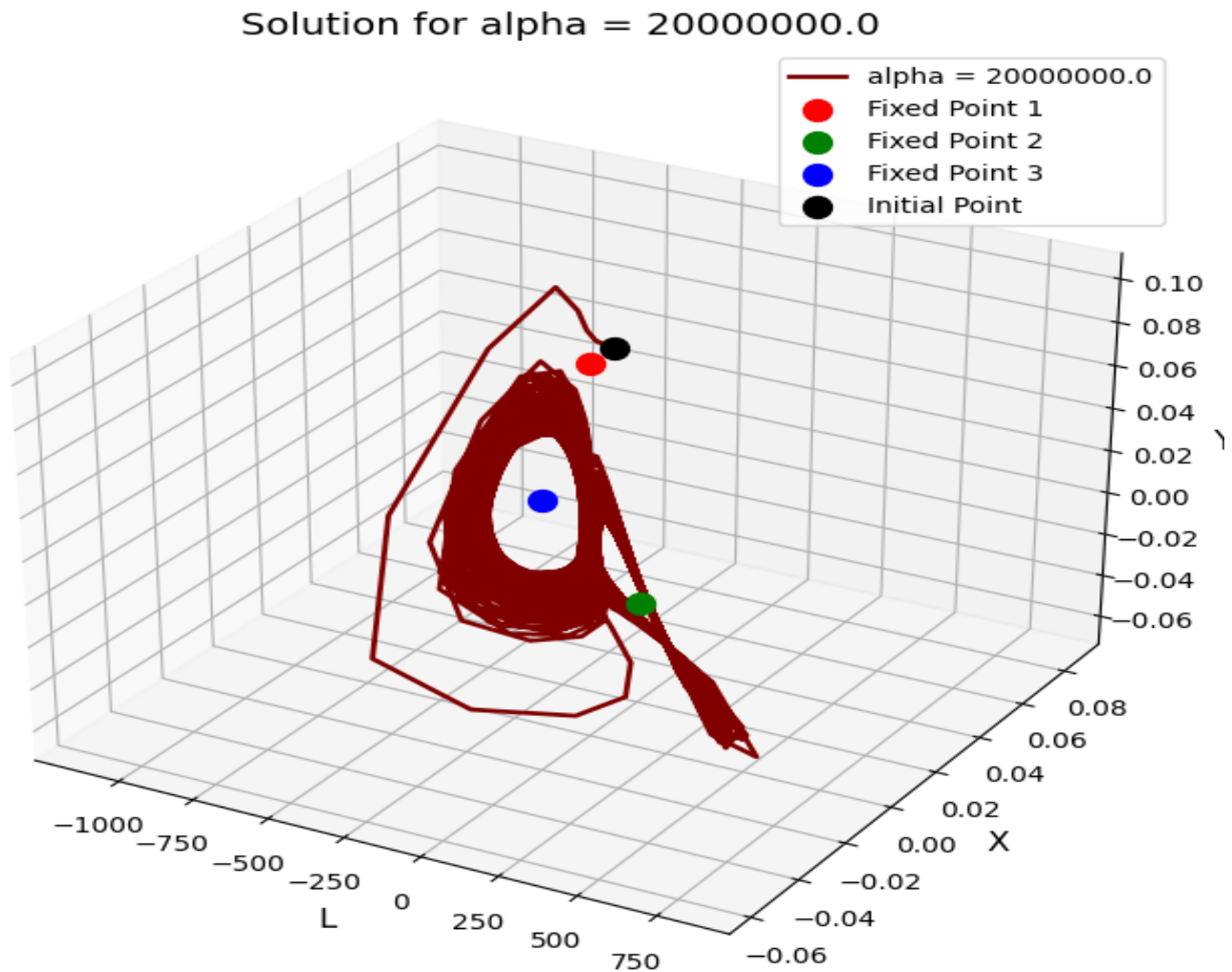


Figure 6: Solution for $\alpha = 2 \times 10^7$.

Behavior: For $\alpha = 2 \times 10^7$, the fixed points (L_2, X_2, Y_2) and (L_3, X_3, Y_3) are significantly further from the origin compared to the other two cases. The trajectories will exhibit even more oscillatory behavior with larger amplitudes. The system takes longer to stabilize and oscillates more significantly before settling near the fixed point 2 (It appears to at least! I am not sure if it is staying there because the perspective is not clear in plot).

Summary:

- As α increases, the fixed points (L_2, X_2, Y_2) and (L_3, X_3, Y_3) move further from the origin. The system exhibits more significant oscillatory behavior and takes longer to stabilize.
- For lower values of α , the system stabilizes more quickly and has lesser oscillations. For higher values, the system has larger oscillations and requires more time to approach the fixed points.

- Since the initial conditions are close to the first fixed point, the trajectories initially tend towards this point before oscillating around and eventually settles near the second or third fixed points depending on the value of α .

3.3 Part (c)

The Jacobian matrix J is given by:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial L} & \frac{\partial f_1}{\partial X} & \frac{\partial f_1}{\partial Y} \\ \frac{\partial f_2}{\partial L} & \frac{\partial f_2}{\partial X} & \frac{\partial f_2}{\partial Y} \\ \frac{\partial f_3}{\partial L} & \frac{\partial f_3}{\partial X} & \frac{\partial f_3}{\partial Y} \end{bmatrix}$$

Using expressions for f_1 , f_2 and f_3 from eq-1, 2 and 3 respectively, we can calculate the partial derivatives as follow:

$$\frac{\partial f_1}{\partial L} = -4800, \quad \frac{\partial f_1}{\partial X} = -4\alpha, \quad \frac{\partial f_1}{\partial Y} = 0$$

$$\frac{\partial f_2}{\partial L} = -5(Y - 0.01), \quad \frac{\partial f_2}{\partial X} = -330, \quad \frac{\partial f_2}{\partial Y} = -5L$$

$$\frac{\partial f_3}{\partial L} = 5X, \quad \frac{\partial f_3}{\partial X} = 5L, \quad \frac{\partial f_3}{\partial Y} = -330$$

Hence, we have:

$$J = \begin{bmatrix} -4800 & -4\alpha & 0 \\ -5(Y - 0.01) & -330 & -5L \\ 5X & 5L & -330 \end{bmatrix}$$

3.4 Part (d)

Following is the code to define Jacobian of matrix and then compute its eigenvalues for different values of α .

```

1 # Defining the Jacobian matrix
2 def jacobian(L, X, Y, alpha):
3     J = np.array([
4         [-4800, -4*alpha, 0],
5         [-5*(Y - 0.01), -330, -5*L],
6         [5*X, 5*L, -330]
7     ])
8     return J
9
10 # Fixed points
11 fixed_points = [(L1, X1, Y1), (L2, X2, Y2), (L3, X3, Y3)]
12
13 # Values of alpha
14 alpha_values = [1e6, 3e6, 2e7]
15
16 # Loop over alpha values
17 for alpha in alpha_values:
18     print("-"*60)
19     print(f"Eigenvalues for alpha = {alpha}:")

```

```

20     print("-"*60)
21
22     for i, point in enumerate(fixed_points):
23         L, X, Y = point
24
25         # Computing the Jacobian matrix
26         J = jacobian(L, X, Y, alpha)
27
28         # Computing eigenvalues
29         eigenvalues = np.linalg.eigvals(J)
30         print(f"Fixed point {i+1}: {eigenvalues} ")
31
32     print("-"*60)
33     print("\n")

```

Eigenvalues for alpha = 1000000.0:

Fixed point 1: [-5171.76 41.76 -330.]
 Fixed point 2: [-4840.73 -309.63 + 1571.69j -309.63 - 1571.69j]
 Fixed point 3: [-4840.73 -309.63 + 1571.69j -309.63 - 1571.69j]

Eigenvalues for alpha = 3000000.0:

Fixed point 1: [-5789.16 659.16 -330.]
 Fixed point 2: [-4919.26 -270.36 + 1634.70j -270.36 -1634.70j]
 Fixed point 3: [-4919.26 -270.36 + 1634.70j -270.36 -1634.70j]

Eigenvalue for alpha = 20000000.0:

Fixed point 1: [-8967.75 3837.75 -330.]
 Fixed point 2: [-5476.22 8.11 + 2036.61j 8.11 - 2036.61j]
 Fixed point 3: [-5476.22 8.11 + 2036.61j 8.11 - 2036.61j]

3.5 Part (e)

For this part I have taken help from [LibreTexts](#)

Eigenvalue Type	Stability	Oscillatory Behavior	Notation
All Real and +	Unstable	None	Unstable Node
All Real and -	Stable	None	Stable Node
Mixed + & - Real	Unstable	None	Unstable Saddle Point
$+a + bi$	Unstable	Undamped	Unstable Spiral
$-a + bi$	Stable	Damped	Stable Spiral
$0 + bi$	Unstable	Undamped	Circle

Table 4: Types of Eigenvalues and Their Stability Characteristics.[8]

For $\alpha = 1 \times 10^6$

- Fixed point 1: Eigenvalues have mixed + and - real parts, indicating instability. This is the reason that the solution never converges to this fixed point.
- Fixed points 2 and 3: They have complex eigenvalues with negative real parts, suggesting stability with damped oscillations.

For $\alpha = 3 \times 10^6$

- Fixed point 1: Eigenvalues have mixed + and - real parts, indicating instability. This is in accordance with solution obtained in part (b). The solution never converges to this point.
- Fixed points 2 and 3: Similar to $\alpha = 1 \times 10^6$, they have complex eigenvalues with negative real parts, suggesting stability with damped oscillations.

For $\alpha = 2 \times 10^7$

- Fixed point 1: Eigenvalues are again mixed and hence not stable.
- Fixed points 2 and 3: They have complex eigenvalues with small real parts, suggesting instability with comparatively lesser damped oscillations. This can be understood from Fig-5 and 6. The oscillation stays for longer in $\alpha = 2 \times 10^7$ case.

Overall, as the α value increases, the stability of the fixed points can change. The behavior of the system is also affected, with some fixed points exhibiting damped oscillations while others show undamped oscillations.

3.6 References

1. Python Documentation: https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html
2. LibreText: [https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_\(Woolf\)/10%3ADynamical_Systems_Analysis/10.04%3A_Using_eigenvalues_and_eigenvectors_to_find_stability_and_solve_ODEs](https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/10%3ADynamical_Systems_Analysis/10.04%3A_Using_eigenvalues_and_eigenvectors_to_find_stability_and_solve_ODEs)

4 Numerical Integration[Dr. Huang]

4.1 Part (a)

An ellipsoid is defined by the equation:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} < 1$$

where a , b , and c are the semi-axis lengths along the x , y , and z axes, respectively.

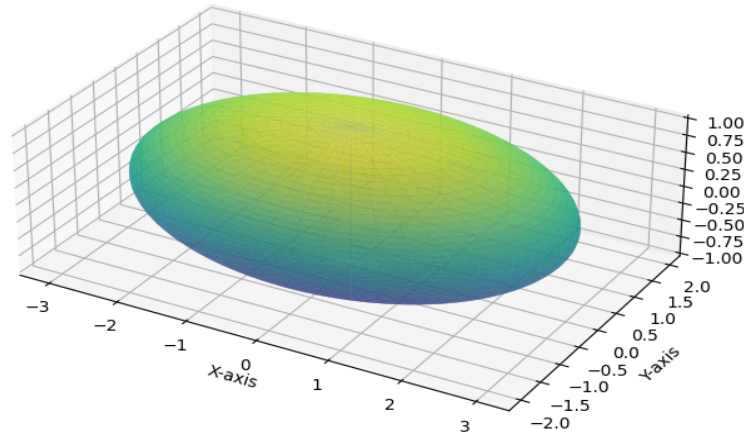


Figure 7: Ellipsoid generated using Python.

The exact volume V_{exact} of the ellipsoid is given by:

$$V_{\text{exact}} = \frac{4}{3}\pi abc$$

The domain to be considered goes from $-a$ to a in x -direction, $-b$ to b in y -direction and $-c$ to c in z -direction. Hence, the volume of the bounding box of the ellipsoid is:

$$V_{\text{box}} = 8abc$$

To estimate the volume using the Monte Carlo method, I am using the following steps:

1. Generate N random points uniformly distributed within the bounding box:

$$x \sim \mathcal{U}(-a, a), \quad y \sim \mathcal{U}(-b, b), \quad z \sim \mathcal{U}(-c, c)$$

where $\mathcal{U}$ denotes a uniform distribution.

2. Count the number of points k that lie inside the ellipsoid:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} < 1$$

3. The estimated volume V_{est} of the ellipsoid is calculated as:

$$V_{\text{est}} = V_{\text{box}} \times \frac{k}{N} = 8abc \times \frac{k}{N}$$

Given the parameters $a = 3.0$, $b = 2.0$, $c = 1.0$, the exact volume of the ellipsoid is:

$$V_{\text{exact}} = \frac{4}{3}\pi(3.0)(2.0)(1.0) = \frac{8}{3}\pi \approx 8.37758$$

For sample sizes $N \in \{1000, 10000, 100000, 1000000, 10000000\}$, the Monte Carlo method provides an estimate V_{est} and the error is:

$$\text{Error} = |V_{\text{exact}} - V_{\text{est}}|$$

N	Calculated Volume	Error	Duration (s)
1000	25.632	0.499259	0.000426526
10000	24.9408	0.191941	0.00368345
100000	25.2514	0.118619	0.0240751
1000000	25.1479	0.0151308	0.183193
10000000	25.138	0.00524277	1.86312

Table 5: Monte Carlo estimation of ellipsoid volume for different N.

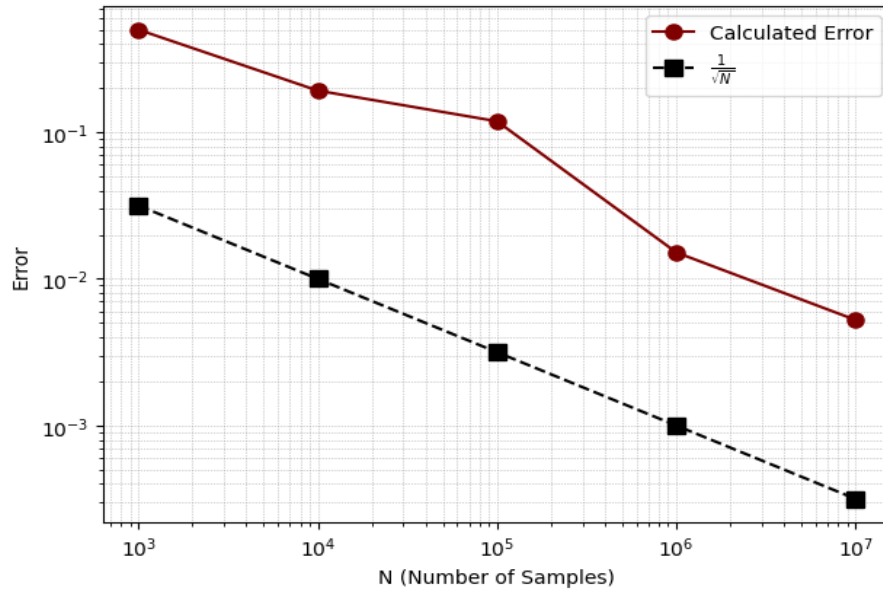


Figure 8: Error vs N plot.

From both the Table-5 and Fig-8, it is clear that error decays as $\frac{1}{\sqrt{N}}$.

4.2 Part (b)

The given integral is:

$$I = \int_{-5}^0 (x^2 + 1) dx \quad (4)$$

To transform the integral domain from $[-5, 0]$ to $[-1, 1]$, we use the change of variable:

$$x = \frac{b-a}{2}t + \frac{b+a}{2}$$

where $a = -5$ and $b = 0$. The differential dx transforms as follows:

$$dx = \frac{b-a}{2} dt$$

Substituting these into the integral, we get:

$$I = \int_{-1}^1 \left(\left(\frac{b-a}{2}t + \frac{b+a}{2} \right)^2 + 1 \right) \frac{b-a}{2} dt \quad (5)$$

To approximate the integral using Gauss-Legendre quadrature, we use the formula:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f \left(\frac{b-a}{2}x_i + \frac{b+a}{2} \right)$$

where x_i are the roots of the Legendre polynomial $P_n(x)$ and w_i are the corresponding weights.

For our integral, we have $a = -5$ and $b = 0$, thus:

$$I \approx \frac{5}{2} \sum_{i=1}^n w_i \left(\left(\frac{5}{2}x_i - \frac{5}{2} \right)^2 + 1 \right)$$

The exact values of x_i and w_i depend on the chosen n , the number of quadrature points.

n	Calculated Integral	Error
1	36.25	10.416699999999999
2	46.666666666666666	3.333333334154531e-05
3	46.666666666666666	3.333333334154531e-05
4	46.666666666666666	3.333333334154531e-05
5	46.666666666666668	3.333333332022903e-05

Table 6: Results of Gauss-Legendre quadrature for different values of n .

As shown in Table-4.2, the approximation of given integral is very accurate for just 2 nodes.

Notes:

- Gauss-Legendre quadrature with n nodes is exact for polynomials of degree $2n - 1$ or less.
 - For the calculation of exact integrals, I have used Integral Calculator.
-

4.3 Part (c)

Given the integral:

$$\int_{-\infty}^{\infty} e^{-(2x-1)^2} (x^2 + 1) dx$$

we need to transform it into a form suitable for Gauss-Hermite quadrature.

Let $u = 2x - 1$. Then:

$$x = \frac{u+1}{2}$$

The differential dx transforms as:

$$dx = \frac{du}{2}$$

Substitute $u = 2x - 1$ and $dx = \frac{du}{2}$:

$$\int_{-\infty}^{\infty} e^{-(2x-1)^2} (x^2 + 1) dx = \int_{-\infty}^{\infty} e^{-u^2} \left(\left(\frac{u+1}{2} \right)^2 + 1 \right) \frac{du}{2}$$

Simplify the integrand:

$$\left(\frac{u+1}{2} \right)^2 + 1 = \frac{(u+1)^2}{4} + 1 = \frac{u^2 + 2u + 1}{4} + 1 = \frac{u^2 + 2u + 1 + 4}{4} = \frac{u^2 + 2u + 5}{4}$$

Thus, the integral becomes:

$$\frac{1}{2} \int_{-\infty}^{\infty} e^{-u^2} \left(\frac{u^2 + 2u + 5}{4} \right) du$$

Factor out constants from the integral:

$$\frac{1}{2} \cdot \frac{1}{4} \int_{-\infty}^{\infty} e^{-u^2} (u^2 + 2u + 5) du = \frac{1}{8} \int_{-\infty}^{\infty} e^{-u^2} (u^2 + 2u + 5) du$$

The integral is now in the form suitable for Gauss-Hermite quadrature:

$$I = \frac{1}{8} \int_{-\infty}^{\infty} e^{-u^2} (u^2 + 2u + 5) du$$

$$\text{with, } f(u) = \frac{1}{8} (u^2 + 2u + 5)$$

n	Calculated Integral	Error
1	1.1077836568159474	0.1107783656815946
2	1.2185620224975422	2.220446049250313e-16
3	1.2185620224975422	2.220446049250313e-16
4	1.2185620224975422	2.220446049250313e-16
5	1.218562022497542	0.0
6	1.218562022497542	0.0

Table 7: Results of Gauss-Hermite quadrature for different values of n

The approximated integral results are again very accurate in this case as well and good results are obtained for just $n=2$.

4.4 References

1. <https://www.integral-calculator.com/>
2. https://en.wikipedia.org/wiki/Gaussian_quadrature
3. ACSI Lecture Notes, Dr. Huang

5 Approximation and Differentiation[Dr. Huang]

5.1 Part (a)

The given objective function is:

$$W = \int_{-1}^1 [f(x) - \sum_{j=0}^n c_j P_j(x)]^2 dx$$

Differentiating W with respect to c_i :

$$\begin{aligned} \frac{\partial W}{\partial c_i} &= \frac{\partial}{\partial c_i} \int_{-1}^1 [f(x) - \sum_{j=0}^n c_j P_j(x)]^2 dx \\ &= \int_{-1}^1 \frac{\partial}{\partial c_i} [f(x) - \sum_{j=0}^n c_j P_j(x)]^2 dx \quad (\text{Leibniz rule for differentiation under the integral sign}) \\ &= 2 \int_{-1}^1 [f(x) - \sum_{j=0}^n c_j P_j(x)] \frac{\partial}{\partial c_i} [f(x) - \sum_{j=0}^n c_j P_j(x)] dx \quad (\text{Using chain rule.}) \\ &= -2 \int_{-1}^1 \left[f(x) - \sum_{j=0}^n c_j P_j(x) \right] P_i(x) dx \quad (\text{Only } P_j = P_i \text{ survived}) \\ &= -2 \int_{-1}^1 \left[f(x) P_i(x) - \left(\sum_{j=0}^n c_j P_j(x) \right) P_i(x) \right] dx \end{aligned}$$

Hence, we have:

$$\boxed{\frac{\partial W}{\partial c_i} = -2 \int_{-1}^1 \left[f(x) P_i(x) - \left(\sum_{j=0}^n c_j P_j(x) \right) P_i(x) \right] dx}$$

5.2 Part (b)

To derive the expression for c_i , we will use the fact that Legendre polynomials of different degrees are orthogonal over $[-1, 1]$ i.e.,

$$\int_{-1}^1 P_n(x) P_m(x) dx = \frac{2\delta_{mn}}{2n+1} \quad \text{where} \quad \delta_{mn} = \begin{cases} 1 & \text{if } m = n, \\ 0 & \text{if } m \neq n. \end{cases}$$

We will use this in the expression obtained in part (a).

$$\begin{aligned} \frac{\partial W}{\partial c_i} &= -2 \left[\int_{-1}^1 f(x) P_i(x) dx - \int_{-1}^1 \left(\sum_{j=0}^n c_j P_j(x) \right) P_i(x) dx \right] \\ &= -2 \left[\int_{-1}^1 f(x) P_i(x) dx - \frac{2c_i}{2i+1} \right] \end{aligned}$$

So, the final expression for $\frac{\partial W}{\partial c_i}$ is:

$$\boxed{\frac{\partial W}{\partial c_i} = -2 \left[\int_{-1}^1 f(x) P_i(x) dx - \frac{2c_i}{2i+1} \right]}$$

To solve for c_i , we can put this derivative equal to zero:

$$\begin{aligned}\frac{\partial W}{\partial c_i} &= -2 \left[\int_{-1}^1 f(x) P_i(x) dx - \frac{2c_i}{2i+1} \right] = 0 \\ \implies \int_{-1}^1 f(x) P_i(x) dx &= \frac{2c_i}{2i+1}\end{aligned}$$

Therefore, the expression for c_i is:

$$c_i = \frac{(2i+1)}{2} \int_{-1}^1 f(x) P_i(x) dx$$

5.3 Part (c)

Given function to be approximated is,

$$f(x) = e^{-x^2} \sin(x-1) + 1$$

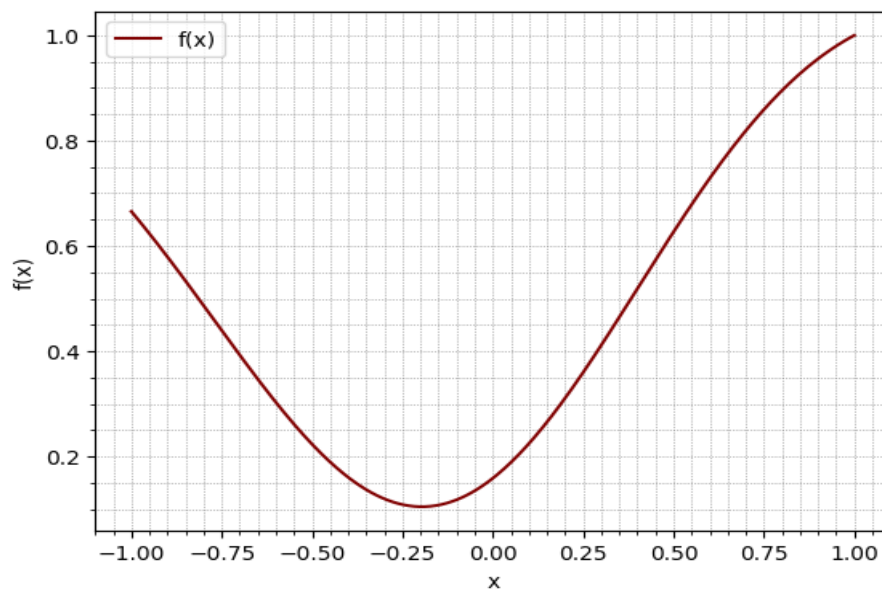
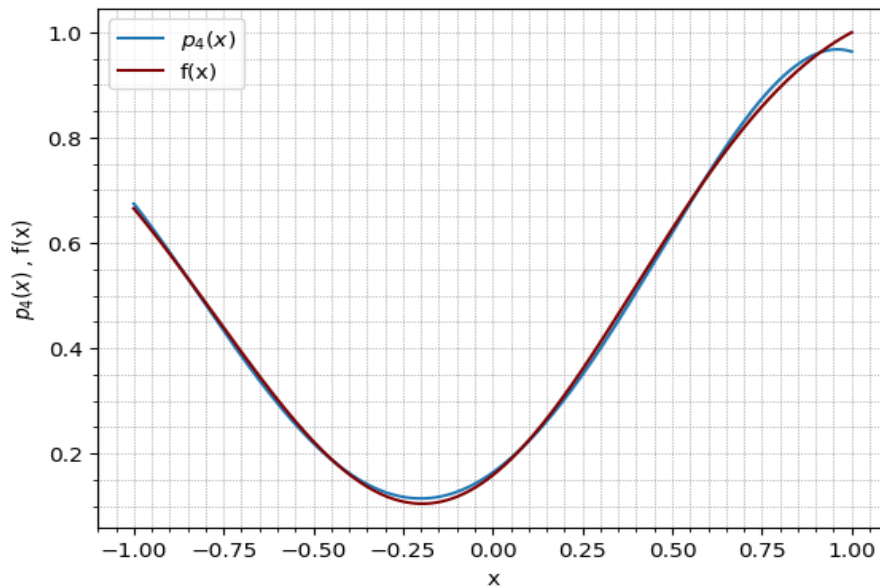


Figure 9: Original function $f(x)$.

Calculation of coefficient c_i :

```
=====
Coefficient c0: 0.44784831278662507
Coefficient c1: 0.28091490992363866
Coefficient c2: 0.4838938841641618
Coefficient c3: -0.13645674773391614
Coefficient c4: -0.11273025713192437
=====
```

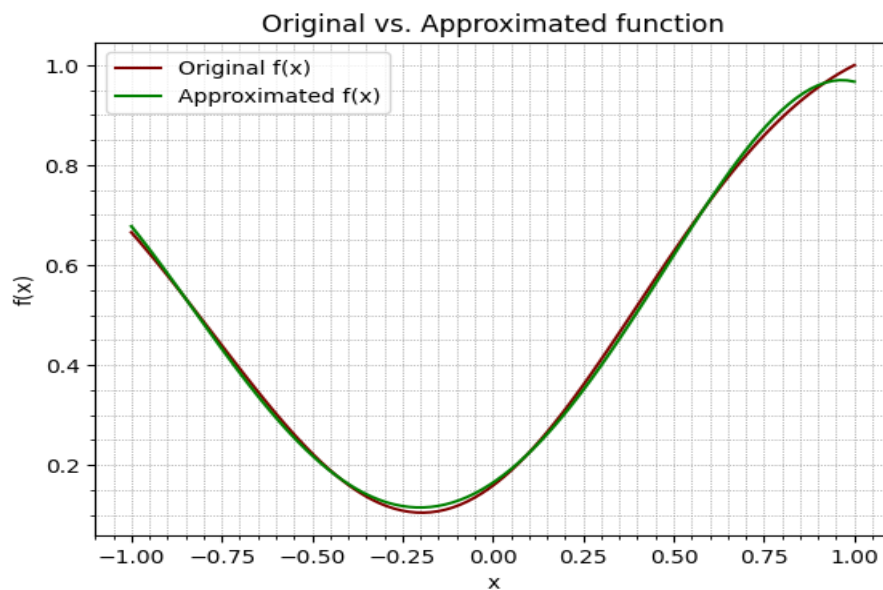
Figure 10: Comparison of $f(x)$ and approximation using Legendre Polynomial.

5.4 Part (d)

Calculation of coefficient c_i :

```
=====
c0: 0.4478686737695029
c1: 0.2809144444406612
c2: 0.4844153372676482
c3: -0.13626722666974048
c4: -0.10999294494482652
=====
```

The resulting coefficient obtained by minimizing W are matching to those obtained in part (c). Only c_4 is slightly different.

Figure 11: Comparison of $f(x)$ and approximated function using Optimization.

5.5 Part (e)

I was getting some warnings in my Python code for this part. I tried a lot and couldn't fix it so I used Matlab for doing this part.

Calculation of coefficient c_i using complex step differentiation:

```
=====
Optimized coefficients c_i:
c0 = 0.44785
c1 = 0.28092
c2 = 0.48389
c3 = -0.13645
c4 = -0.11273
=====
```

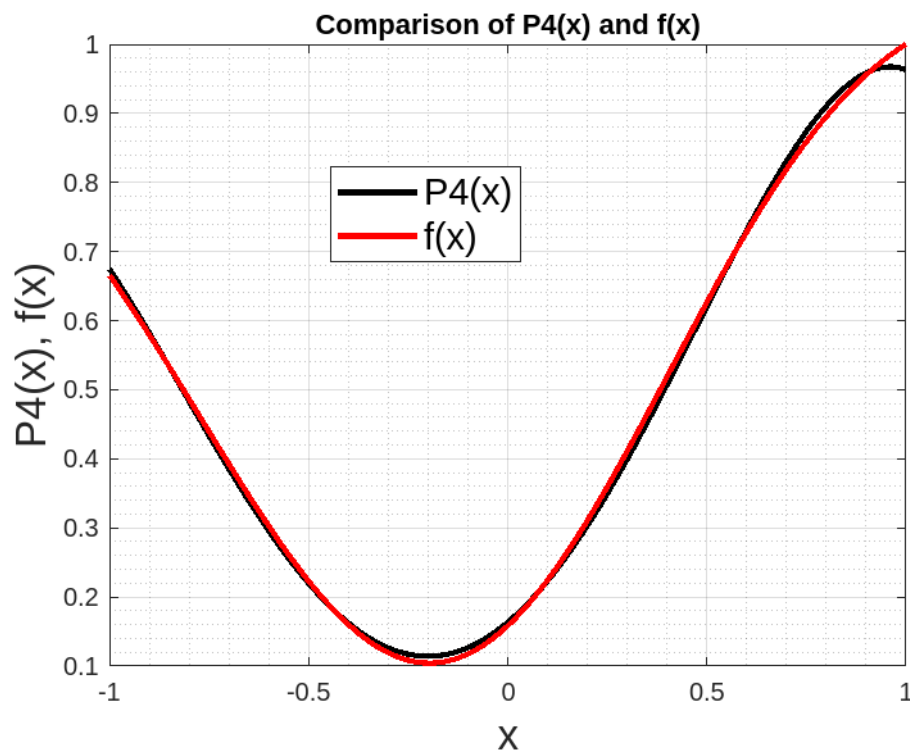


Figure 12: Comparison of approximated function and original function.

NOTES:

- I have used `scipy.integrate.quad` for integration.
- For optimization (part d), BFGS method from `scipy.optimize` is used.
- For part (e), the Matlab code is used and file name is *Q5-e.m*

5.6 References

1. ACSI Lecture Notes and codes, Dr. Huang
2. <https://math.stackexchange.com/questions/2147167/>

6 Data Structures[Dr. Erlebacher]

Not Attempted.

7 Maximum Likelihood Estimation[Dr. Chipilski]

7.1 Part (a)

The given probability density function (pdf) is:

$$p(x_i; \mu, \sigma^2) = \frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right)$$

It is given that the samples are *iid*(independent and identically distributed), so we can express the joint pdf or the Likelihood function as the product of individual pdfs of each of these observations. If there are n observations, the joint pdf can be written as:

$$p(x_1, \dots, x_n; \mu, \sigma^2) = \prod_{i=1}^n p(x_i; \mu, \sigma^2)$$

Substituting the expression for $p(x_i; \mu, \sigma^2)$, we get:

$$p(x_1, \dots, x_n; \mu, \sigma^2) = \prod_{i=1}^n \frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right)$$

This is our likelihood function.

$$L(\theta; x_1, \dots, x_n) = \prod_{i=1}^n \frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right) \quad (6)$$

7.2 Part (b)

Taking the natural logarithm of the joint pdf in eq.6, we get the log-likelihood function $\ell(\theta; x_1, \dots, x_n)$:

$$\begin{aligned} \ell(\theta; x_1, \dots, x_n) &= \ln L(\theta; x_1, \dots, x_n) \\ &= \ln \left[\prod_{i=1}^n \frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^n \ln \left[\frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right) \right] \quad (\text{using } \ln(ABC\dots) = \ln A + \ln B + \ln C + \dots) \\ &= \sum_{i=1}^n \left[\ln \frac{1}{x_i \sqrt{2\pi\sigma^2}} + \ln \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^n \left[-\ln x_i \sqrt{2\pi\sigma^2} + \ln \exp\left(-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^n \left[-\ln(x_i) - \frac{1}{2} \ln(2\pi\sigma^2) - \frac{(\ln x_i - \mu)^2}{2\sigma^2} \right] \\ &= \sum_{i=1}^n \left[-\ln(x_i) - \frac{1}{2} \ln(2\pi) - \frac{1}{2} \ln(\sigma^2) - \frac{(\ln x_i - \mu)^2}{2\sigma^2} \right] \\ &= -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n [\ln(x_i) - \mu]^2 - \sum_{i=1}^n \ln(x_i) \end{aligned}$$

Here, I have used the properties of log and the fact that when a constant term is summed n times, the result can be obtained by multiplying it by n . Hence, we have:

$$\ell(\theta; x_1, \dots, x_n) = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n [\ln(x_i) - \mu]^2 - \sum_{i=1}^n \ln(x_i)$$

7.3 Part (c)

The final expression in last part represents the log-likelihood function for the given data and parameters. To find the maximum likelihood estimates (MLEs) of μ and σ^2 , we can differentiate this expression with respect to μ and σ^2 , set the derivatives equal to zero, and solve for μ and σ^2 .

To find $\hat{\mu}$, differentiating l with respect to μ , we get:

$$\frac{\partial l}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n [\ln(x_i) - \mu]$$

Setting this equal to zero:

$$\begin{aligned} \frac{1}{\sigma^2} \sum_{i=1}^n [\ln(x_i) - \hat{\mu}] &= 0 \\ \implies \sum_{i=1}^n [\ln(x_i)] - n\hat{\mu} &= 0 \end{aligned}$$

$$\implies \hat{\mu} = \frac{1}{n} \sum_{i=1}^n \ln(x_i)$$

To find $\hat{\sigma}^2$, we can differentiate l with respect to σ^2 :

$$\frac{\partial l}{\partial (\sigma^2)} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=1}^n [\ln(x_i) - \mu]^2$$

Setting this equal to zero:

$$-\frac{n}{2\hat{\sigma}^2} + \frac{1}{2(\hat{\sigma}^2)^2} \sum_{i=1}^n [\ln(x_i) - \hat{\mu}]^2 = 0$$

$$\text{On rearranging, we have: } n = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^n [\ln(x_i) - \hat{\mu}]^2 = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^n [\ln(x_i) - \hat{\mu}]^2$$

Finally, we obtain the MLE expressions for $\hat{\sigma}^2$:

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n [\ln(x_i) - \hat{\mu}]^2 = \frac{1}{n} \sum_{i=1}^n \left[\ln(x_i) - \frac{1}{n} \sum_{i=1}^n \ln(x_i) \right]^2$$

Complete MLE expression:

$$\hat{\theta} = [\hat{\mu} \quad \hat{\sigma}^2]^T = \left[\frac{1}{n} \sum_{i=1}^n \ln(x_i) \quad \frac{1}{n} \sum_{i=1}^n \left(\ln(x_i) - \frac{1}{n} \sum_{i=1}^n \ln(x_i) \right)^2 \right]^T$$

7.4 Part (d)

The obtained results are

MLE estimate of Mean 2.95474

MLE estimate of Variance: 0.97423

Hence, $\hat{\theta} = [\hat{\mu} \ \hat{\sigma}^2]^T = [2.95474 \ 0.97423]^T$

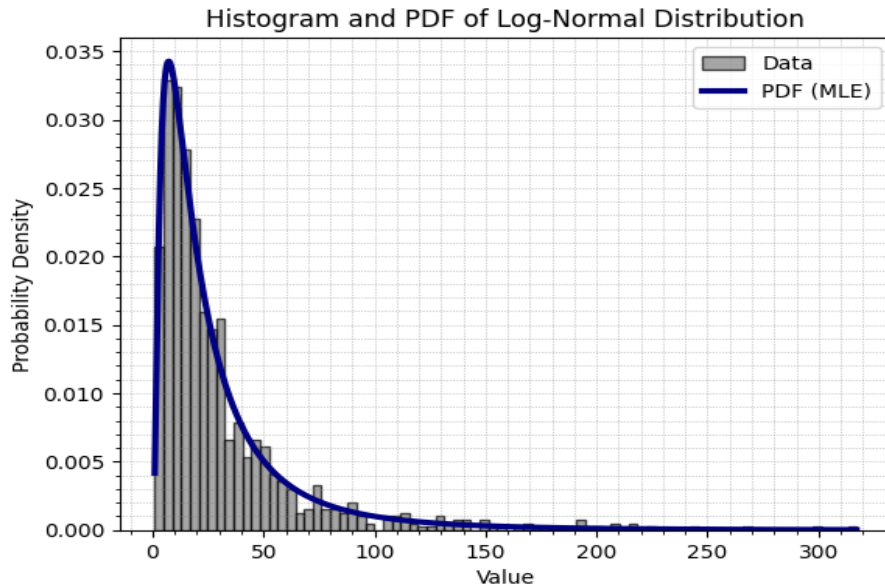


Figure 13: Histogram of *lognormal_sample.txt* with MLE pdf overlaid.

The MLE pdf is displaying the same behaviour to that of data. This shows that the calculated MLE expression obtained is correct and a good fit for the data.

7.5 Part (e)

The obtained results using gradient descent optimization used are:

RESULTS

Iterations: 89

mu: 2.95474

sigma: 0.97424

Hence, $\hat{\theta} = [\hat{\mu} \ \hat{\sigma}^2]^T = [2.95474 \ 0.97424]^T$

It takes 89 iterations to converge to the analytical MLE estimates calculated in part (d). As shown in Fig-14, $\tilde{l}$ and μ converges smoothly from their initial guesses. However, σ^2 first deviates and starts increasing and then decreases to converged value. This might be because of sub optimal initial guess

of direction. Also, the convergence depend greatly on initial values and step-size α . For good initial guesses (like in this case), the solution will converge in lesser number of iterations.

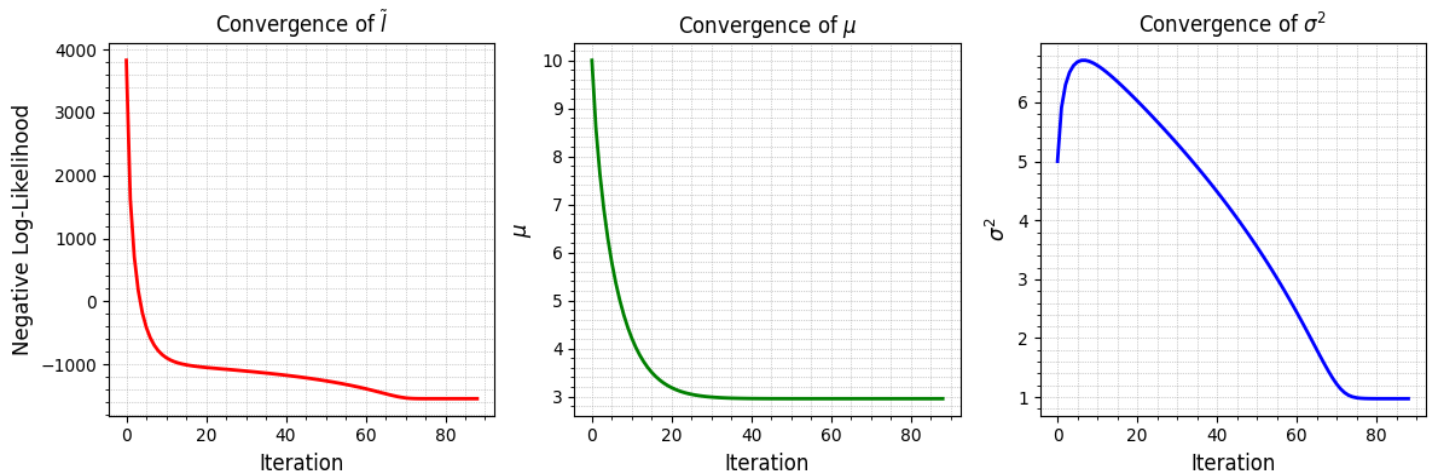


Figure 14: Numerical results of MLE

7.6 References

1. ACSII Lecture Notes, Dr. Plewa
2. Data Assimilation Lecture Notes, Dr. Chipilski
3. Python documentation: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.lognorm.html>
4. Gradient descent algorithm: https://en.wikipedia.org/wiki/Gradient_descent

8 Parallel Programming[Dr. Zavala]

8.1 OpenMP

The function to be integrated is

$$h(x) = e^x$$

Simpson integration can be used to approximate this integral. The formula is given by:

$$I \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) + f(x_n) \right]$$

Results for n=1000:

Threads	Integral	Error	Time (microseconds)
1	1.71828	6.66134×10^{-15}	35
2	1.71828	3.77476×10^{-15}	105
4	1.71828	2.26485×10^{-15}	88
6	1.71828	6.43929×10^{-15}	122
8	1.71828	5.55112×10^{-15}	89
10	1.71828	4.88498×10^{-15}	76
16	1.71828	4.44089×10^{-16}	115

Table 8: Integration Results with $n = 10^3$

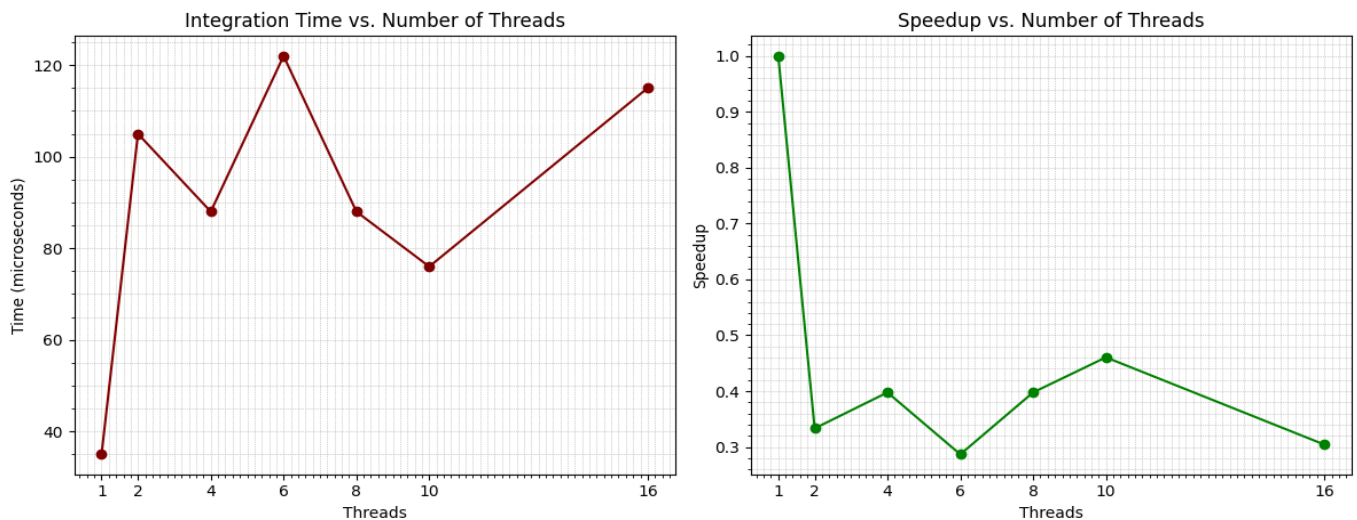


Figure 15: Performance of the OpenMp code with $n = 10^3$ sections

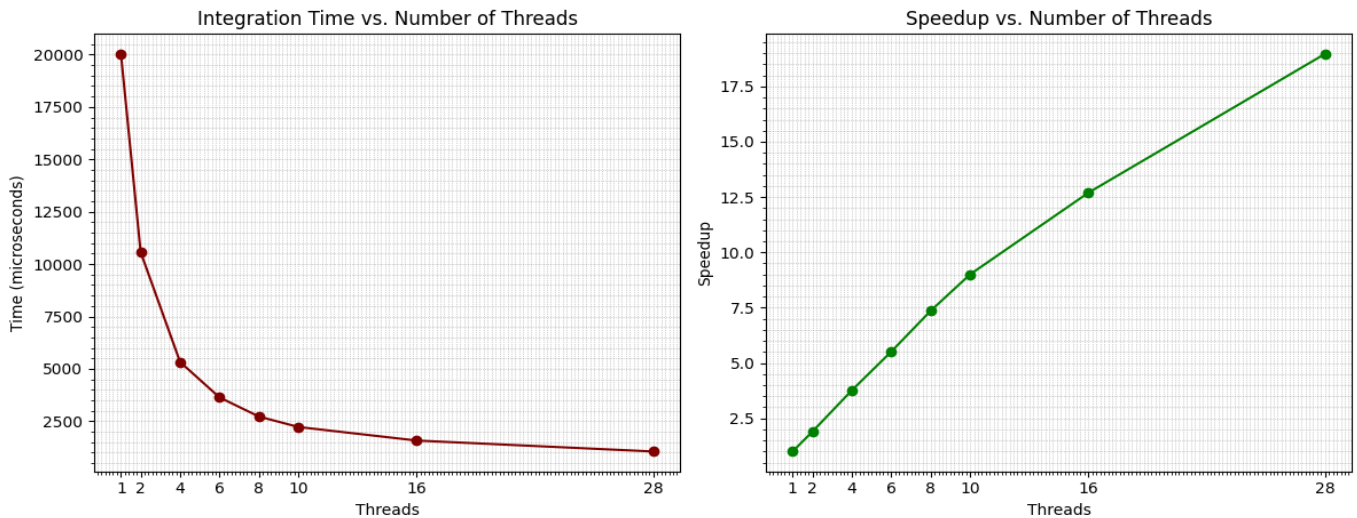
Discussion and Explanation: As clear from Table-8 and Fig-15, the speedup of the code is very poor. With increase in number of threads, the code shows inconsistent behaviour. The reason for this poor performance is [Parallelization Overhead](#) which is occurring because of very small computation workload per thread. For $n = 1000$, the overhead of creating and managing multiple threads is outweighing the benefits of parallelization, leading to suboptimal performance.

The code gives very good speedup for larger values of n . For demonstration, I have done the calculations with $n = 1000000$ sections.

Results for $n=1000000$:

As shown in Table-9 and Fig-16, the speedup obtained for $n=1000000$ is very good (almost linear). In this case, the benefits of parallelization are outweighing the overhead of creating and managing multiple threads.

Threads	Integral	Error	Time (microseconds)
1	1.71828	6.66134×10^{-15}	20035
2	1.71828	3.77476×10^{-15}	10570
4	1.71828	2.26485×10^{-15}	5327
6	1.71828	6.43929×10^{-15}	3613
8	1.71828	5.55112×10^{-15}	2723
10	1.71828	4.88498×10^{-15}	2225
16	1.71828	4.44089×10^{-16}	1578
28	1.71828	4.44089×10^{-16}	1056

Table 9: Integration Results with $n = 10^6$ Figure 16: Performance of the OpenMp code with $n = 10^6$ sections**NOTES:**

- I have used classroom machines to run OpenMP codes. There are 28 threads in total on those machines. A simple code `max_threads.cpp` is used to get the number of threads.
- The openMP implementation is present in `8_openmp.cpp`. This code uses a function file `simpson.h` which I already had from Assignment 12 (FALL23) Scientific Programming course.
- **Compilation Commands used:**

```
-----
[mt22t@class106 Q8-200973108]$ g++ -fopenmp 8_openmp.cpp -o openmp
[mt22t@class106 Q8-200973108]$ ./openmp
-----
```

8.2 MPI

I got a lot of bugs in my initial code and I used chatGpt for debugging them. I have attached the conversation at the end.

Code Implementation Details

1. **MPI Initialization** : MPI is initialized using `MPI_Init` function, and the rank and size of the MPI processes are obtained using `MPI_Comm_rank` and `MPI_Comm_size` functions, respectively.
2. **Division of Interval** The interval $[0, 1]$ is evenly divided into $2N_p$ subintervals, where N_p is the number of processes.
3. **Local Integration** Each process computes its local integral using Simpson's rule for the assigned subintervals.
4. **Ring-like Communication** Processes exchange function values in a ring-like fashion using `MPI_Send` and `MPI_Recv` functions.
5. **Gathering Results** The local results are gathered on process 0 using `MPI_Gather` function.

Process	Area (sq. units)
0	0.104908
1	0.115941
2	0.128135
3	0.141611
4	0.156504
5	0.172964
6	0.191155
7	0.211259
8	0.233477
9	0.258032
Full approx.	1.71399

Table 10: Integration Results using MPI.

NOTES:

- I have used classroom machines to run MPI codes.
- The MPI implementation is present in `8_MPI.cpp`. This code is based on [Moreno Marzolla's](#) code.
- **Compilation Commands used:**

```
-----
[mt22t@class106 Q8-200973108]$ mpic++ 8_MPI.cpp -o MPI
[mt22t@class106 Q8-200973108]$ mpirun -np 10 MPI
-----
```

8.3 References

1. Scientific Programming Assignment 12, Dr. Erlebacher
2. High Performance computing Lecture Notes on OpenMP, Dr. Wang
3. ChatGpt conversation: <https://chatgpt.com/share/afee595c-0ed8-4257-9279-eb0cfbf55252>
4. <https://www.moreno.marzolla.name/teaching/HPC/handouts/mpi-ring.html>

9 Statistics[Dr. Beerli]

9.1 Part 1

To get an idea of the growth model to use, I first plotted the time series of bacterial growth. Before this I first asked ChatGpt(conversation is [attached](#)) about the potential growth models which can be used and under what cases can they be used. The different growth models which are commonly used for bacterial growth are:

1. **Exponential Growth Model:** Appropriate when bacterial growth is in exponential phase.
2. **Gompertz Growth Model:** Appropriate when bacterial growth shows a sigmoidal growth pattern.
3. **Monod Model:** Appropriate when the growth rate depends on the concentration of a limiting nutrient.
4. **Logistic Growth Model:** Appropriate when the growth rate has a saturation point.

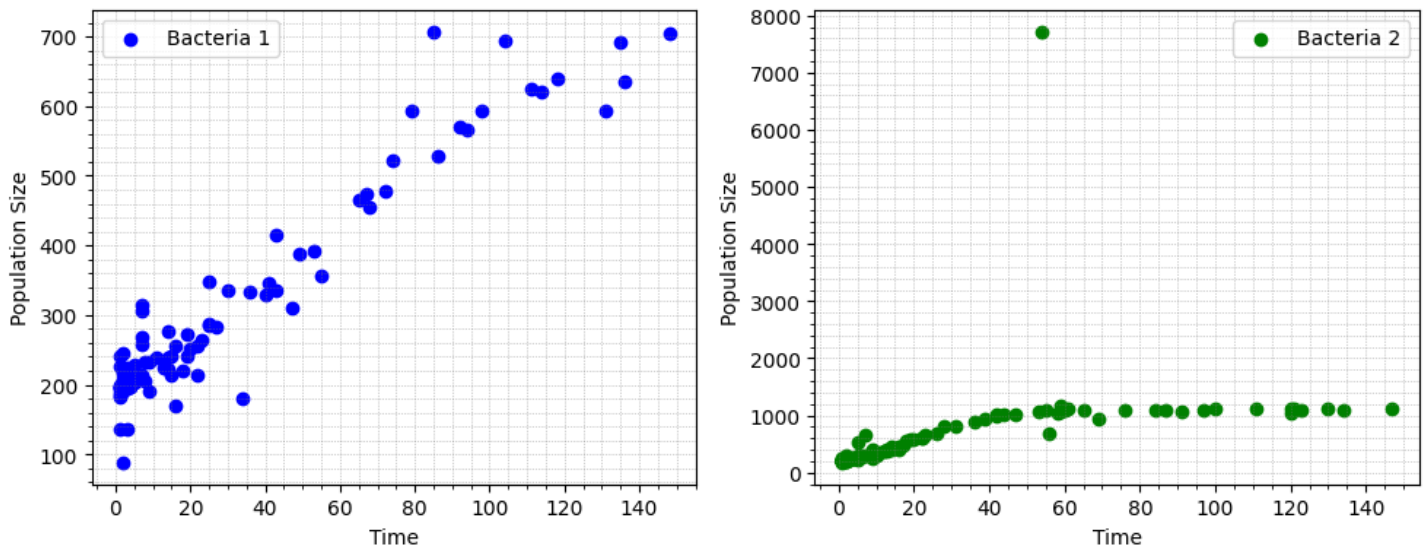


Figure 17: Plots of given data as a function of time.

As clear from the Fig-17, the data is varying logarithmically and hence Logistic Growth model should be used. Another reason for choosing Logistic Growth model is that the plots for sure doesn't show exponential growth behaviour and I am not familiar with the remaining two models. For simplicity, I am taking two parameters only in this model. This was also suggested by ChatGpt.

Logistic Growth model:

The logistic growth model describes how a population grows over time in an environment with limited resources. It is a commonly used model in ecology to represent the growth of populations, including bacterial growth. The model is characterized by an initial exponential growth phase, followed by a slowdown as the population approaches the carrying capacity of the environment.

$$N(t) = \frac{K}{1 + \left(\frac{k - N_0}{N_0} \right) e^{-rt}}$$

where:

- $N(t)$ is the population size at time t .
- N_0 is the initial population size.
- K is the carrying capacity (parameter 1).
- r is the intrinsic growth rate (parameter 2).

9.2 Part 2

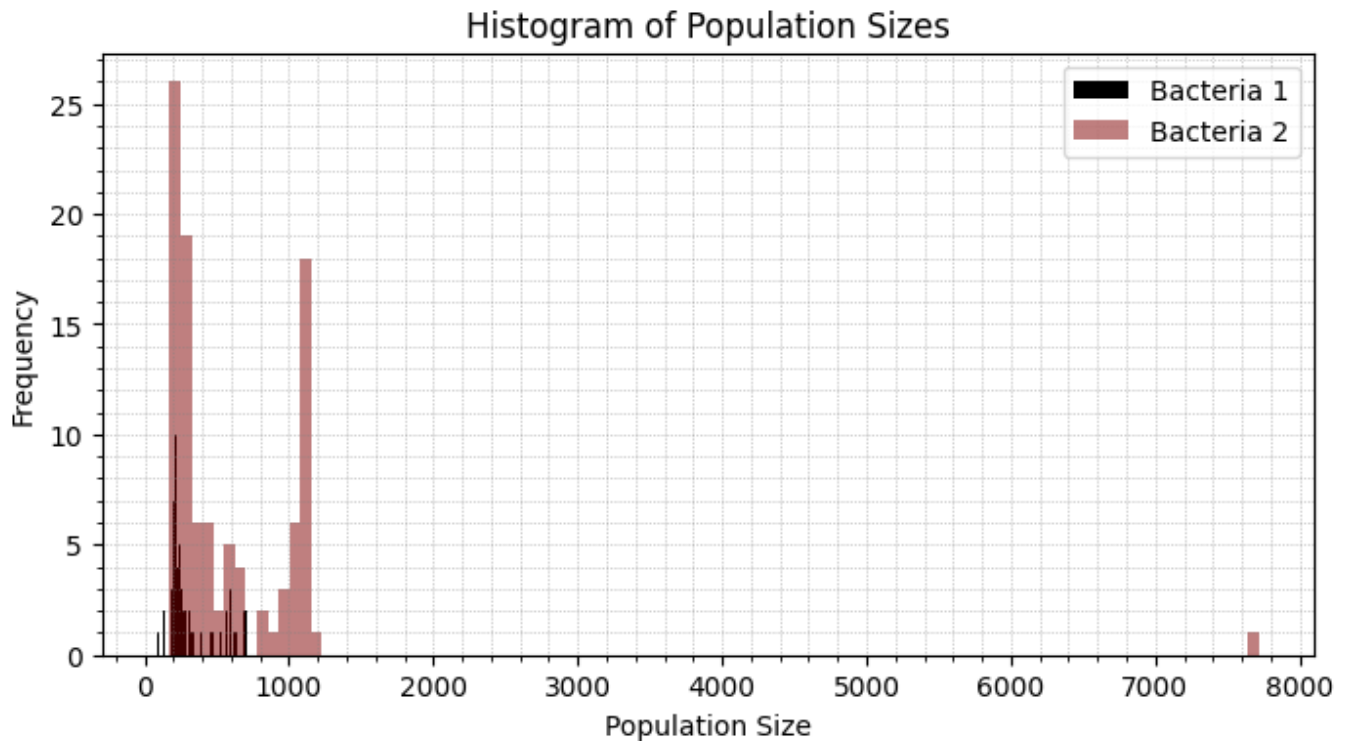


Figure 18: Histogram of both bacteria for comparison.

As clear from the Fig-18 above, the growth of bacteria 2 is more spread out and has higher population size compared to bacteria 1 which is having population size at a narrower range. The population of bacteria 1 stays in range of 100 to 700. Whereas the population of bacteria ranges approximately between 200 to 1200. One more interesting point is the sudden jump in population of bacteria 2 at $t = 54$. The population suddenly becomes around 7000 which is also shown in the histogram. I don't know if this sudden jump in population is possible in reality or not. This may be because some error in data collection.

9.3 Part 3

I have used Wikipedia page [\[6\]](#) and Youtube [video](#) to study about Metropolis-hastings scheme. ChatGpt helped me to debug my code.

Metropolis Hastings Algorithm:

```

1 Input:
2 - Initial state  $x_0$ 
3 - Proposal distribution  $q(y|x)$ 

```

```

4 - Target distribution p(x)
5 - Number of iterations
6
7 Output:
8 - Samples from the target distribution
9
10 Initialize the Markov chain at an arbitrary state x_0.
11 For each iteration t = 1, 2, ...:
12     Generate a proposal state y from the proposal distribution q(y|x_{t-1}).
13     Calculate the acceptance probability:
14         alpha = min(1, p(y) * q(x_{t-1}|y) / (p(x_{t-1}) * q(y|x_{t-1})))
15     Generate a uniform random number u from the interval [0, 1].
16     If u <= alpha, accept the proposal and set x_t = y; otherwise, reject
        the proposal and set x_t = x_{t-1}.
17 Repeat the above steps for a sufficient number of iterations to obtain a
        representative sample from the target distribution.

```

I varied my initial parameters and iterations until I got results in the credible intervals. For confidence interval of 95%, I got the following results with 1000000 iterations in the algorithm. For the remaining question I used $k = 1000$ and $r = 0.1$ as initial guess of my parameters.

Results:

```

=====
Posterior Summary Statistics:
-----

Bacteria 1:
K: Mean = 740.64, Std = 133.05, Credible Interval = [626.15, 985.07]
r: Mean = 0.04, Std = 0.01, Credible Interval = [0.03, 0.05]
-----

Bacteria 1:
K: Mean = 1279.1786, Std = 27.0274, Credible Interval = [1181.4383, 1285.6450]
r: Mean = 0.1197, Std = 0.0019, Credible Interval = [0.1192, 0.1264]
=====

```

Once, I got the final parameters within credible intervals, I plotted the results with raw data. Following are the results I am getting. They are within the credible intervals defined. Fig-19 shows the evolution of k and r for 10^6 iterations. For calculation of final statistics first 10^3 iterations are discarded.

```

-----
Parameters for Bacteria 1:
-----
Mean K: 740.635076632971
Mean r: 0.041418087653550005
Final K: 626.5084346930691
Final r: 0.049222993499177116
-----

Parameters for Bacteria 2:
-----
Mean K: 1279.1786095887194
Mean r: 0.1197049948309464
Final K: 1284.8533881381127
Final r: 0.1193281096824759

```

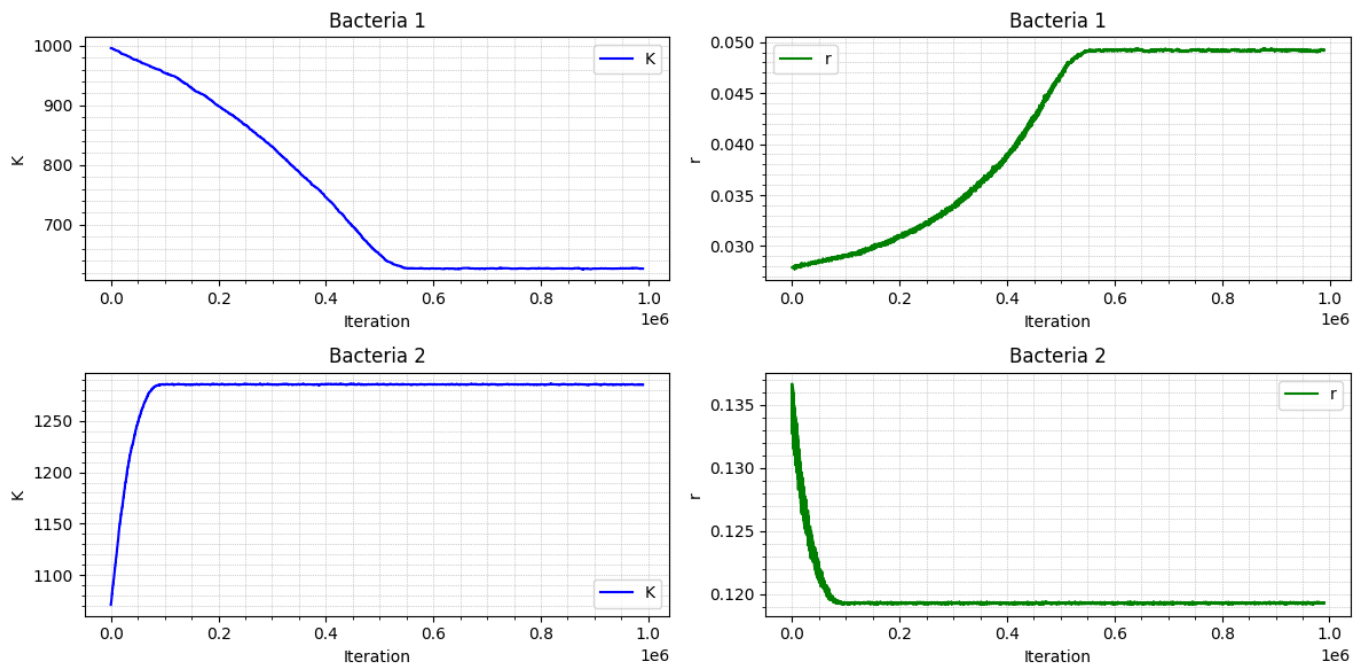


Figure 19: Growth of Parameters K and r with iteration for Metropolis Hastings Algorithm.

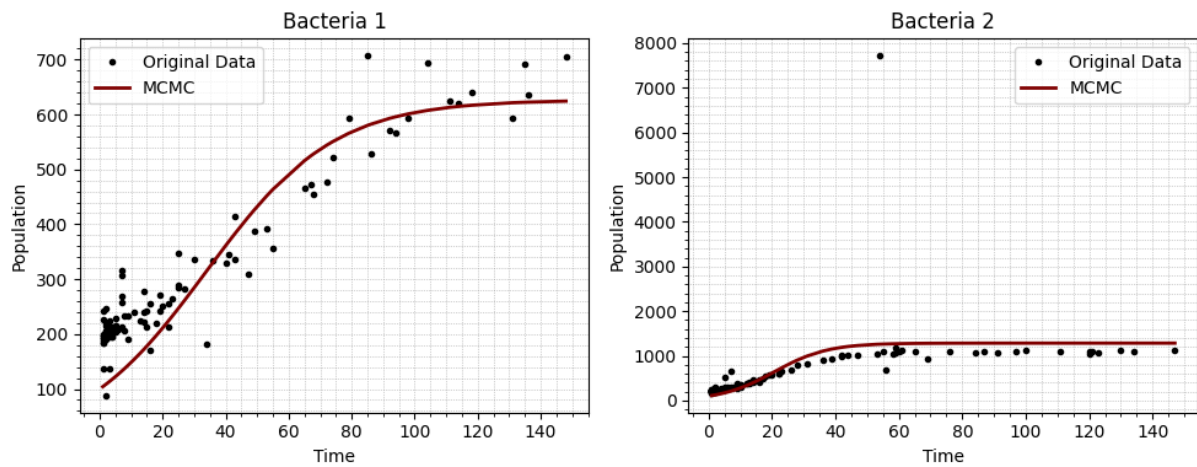


Figure 20: Comparison of obtained model with parameters estimated using Metropolis-Hastings and raw data from the two datasets- bacterial1.txt and bacteria2.txt

9.4 Part 4

I have tried using two methods. Initially, I was trying to implement Expectation-minimization algorithm, but due to some issues it was underestimating parameters and the values were not within credible intervals as well. I was not able to figure out issues even with the help of AI. So, I implemented Particle Swarm Optimization. I used the Wikipedia page [7] for Particle Swarm Optimization to read about it and I took the code from [tutorial](#) given on GeeksforGeeks[3].

Particle Swarm Optimization Algorithm:

```

1 Input:
2   - t: Time points
3   - population: Observed population sizes
4   - num_particles: Number of particles (default: 30)

```

```

5  - max_iter: Maximum iterations (default: 10000)
6  - w: Inertia weight (default: 0.5)
7  - c1: Cognitive parameter (default: 1)
8  - c2: Social parameter (default: 2)
9
10 Output:
11   - Optimal parameters (K, r)
12
13 Initialize swarm with random positions and velocities
14 Initialize personal and global best positions and scores
15
16 For each iteration:
17     For each particle:
18         Update velocity:
19             velocity = w * velocity + c1 * rand() * (personal_best -
20             position) + c2 * rand() * (global_best - position)
21         Update position: position += velocity
22         Evaluate fitness: mean squared error (MSE)
23         Update personal best if fitness improves
24         Update global best if any personal best improves
25
26 Return global best position (optimal K and r)

```

Results:

Final K and r values for Bacteria 1:

K: 626.7679919860823

r: 0.049196367822090185

Final K and r values for Bacteria 2:

K: 1285.298819322784

r: 0.11928223995070214

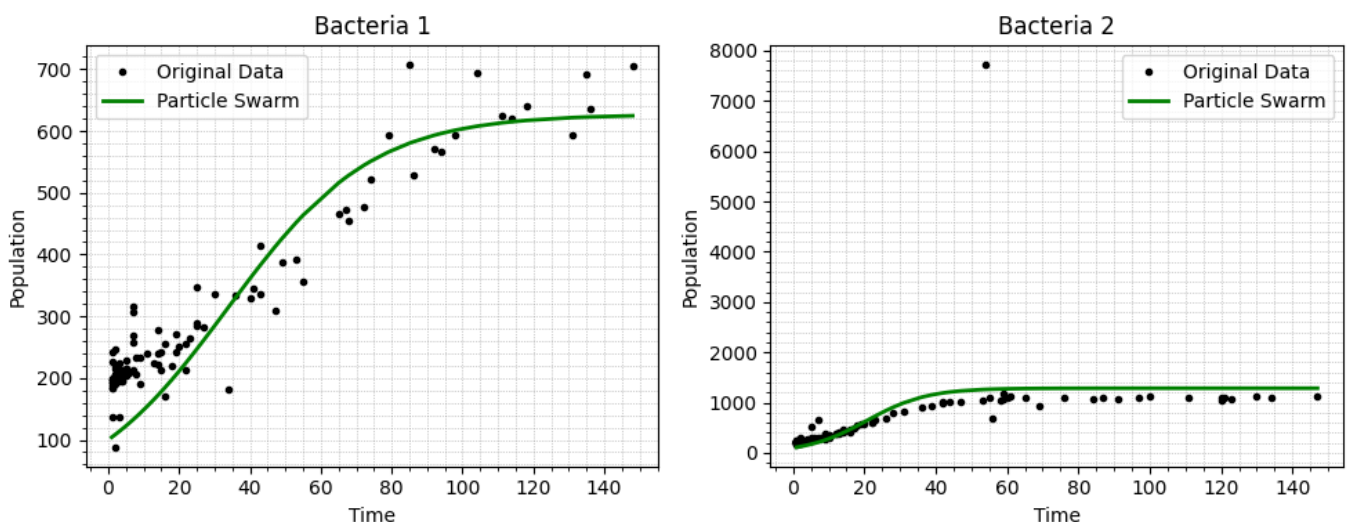


Figure 21: Comparison of obtained model with parameters estimated using the three algorithms and raw data from the two datasets- bacterial1.txt and bacterial2.txt

9.5 Part 5

The parameters estimated using Particle swarm optimization and Metropolis-Hastings are almost same suggesting that the model works well. Also, these parameters lie within the support interval. They fit the model very well which can be observed from Fig-22. However, there is some issue with Expectation minimization method and it is not able to generate good results. Table-11 compares the parameter obtained using three methods and also gives the support intervals.

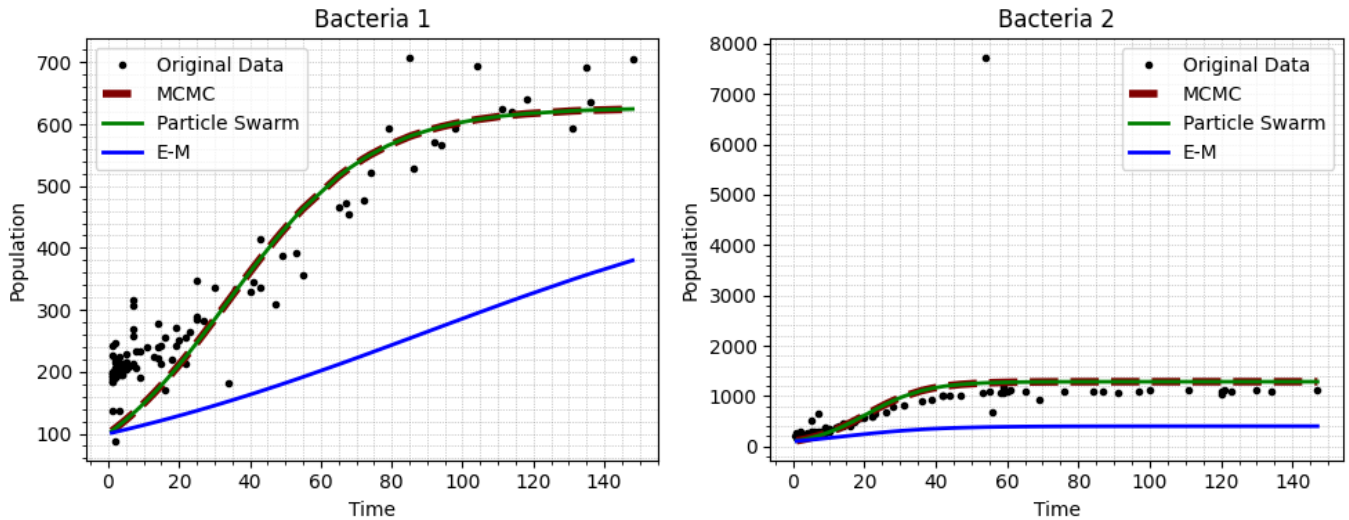


Figure 22: Comparison of obtained model with parameters estimated using Particle Swarm Optimization and raw data from the two datasets- bacteria1.txt and bacteria2.txt

Bacteria 1				
Parameter	MCMC	PSO	EM	Support Interval
K	626.51	626.77	527.50	(626.15, 985.07)
r	0.05	0.05	0.02	(0.0300, 0.0500)
Bacteria 2				
Parameter	MCMC	PSO	EM	Support Interval
K	1284.78	1285.30	401.86	(1181.4383, 1285.6450)
r	0.12	0.12	0.08	(0.1192, 0.1264)

Table 11: Parameter values for Bacteria 1 and Bacteria 2 from MCMC, PSO, and EM methods and credible intervals.

The results here suggests that the logistic Growth model adopted at the beginning was correct.

9.6 References

- [1] ChatGpt conversation: <https://chatgpt.com/share/81c491ac-536e-452d-a08c-2adf6d260d20>
- [2] YouTube: https://www.youtube.com/watch?si=Sp70nRXv1PiVzHKb&v=MxI78mpq_44&feature=youtu.be

10 Numerical Integration[Dr. Sachin]

10.1 Part (a)

The given function to be integrated is

$$f_n(x) = \sum_{j=1}^n \frac{|\sin(j\pi x)|}{j}$$

This function is continuous over the domain $x \in [0, 1]$ but its derivative $f'_n(x)$ is discontinuous.

Points of discontinuity of $f'_n(x)$:

Differentiating $f_n(x)$ with respect to x :

$$\begin{aligned} f'_n(x) &= \frac{d}{dx} \left(\sum_{j=1}^n \frac{|\sin(j\pi x)|}{j} \right) \\ f'_n(x) &= \sum_{j=1}^n \frac{d}{dx} \left(\frac{|\sin(j\pi x)|}{j} \right) \end{aligned}$$

Now, chain rule can be used to calculate the derivative of $|\sin(j\pi x)|$:

$$\begin{aligned} \frac{d}{dx} |\sin(j\pi x)| &= \frac{d}{d(\sin(j\pi x))} |\sin(j\pi x)| \cdot \frac{d}{dx} (\sin(j\pi x)) \\ &= \text{sgn}(\sin(j\pi x)) \cdot (j\pi) \cdot \cos(j\pi x) \end{aligned}$$

where $\text{sgn}(x)$ is the sign function defined as:

$$\text{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Thus, the derivative of $f_n(x)$ becomes:

$$f'_n(x) = \sum_{j=1}^n \frac{\text{sgn}(\sin(j\pi x)) \cdot (j\pi) \cdot \cos(j\pi x)}{j}$$

Hence,

$$f'_n(x) = \pi \sum_{j=1}^n \text{sgn}(\sin(j\pi x)) \cdot \cos(j\pi x)$$

The derivative $f'_n(x)$ will be discontinuous where the sign of $\sin(j\pi x)$ changes. One such point is $x = 0$ whose discontinuity is straightforward to show (I have done that at the end in **Appendix**). Other points of discontinuity occur when x crosses multiples of $\frac{1}{j}$, for integer values of j . So, the points of discontinuity of $f'_n(x)$ are at the points:

$$x = \frac{k}{j}$$

where k is an integer and j ranges from 1 to n .

For $n = 30$, the number of such discontinuities can be calculated as follow:

For each j from 1 to n , we need to count how many valid k there are such that $0 \leq \frac{k}{j} \leq 1$:

- For $j = 1$, k can be 0 or 1. This gives 2 points.
- For $j = 2$, k can be 0, 1, 2. This gives 3 points.
- For $j = 3$, k can be 0, 1, 2, 3. This gives 4 points.
- ...
- For $j = n$, k can be 0, 1, 2, ..., n . This gives $n + 1$ points.

In general, for a given j , the number of discontinuity points is $j + 1$.

The total number of points is the sum of all discontinuity points from $j = 1$ to n :

$$\sum_{j=1}^n (j + 1) = \sum_{j=1}^n j + \sum_{j=1}^n 1$$

This is easy to calculate and the summation is equal to 495. So, there are 495 points of discontinuity of $f'_n(x)$. The same is verified using the Python code in 10.ipynb.

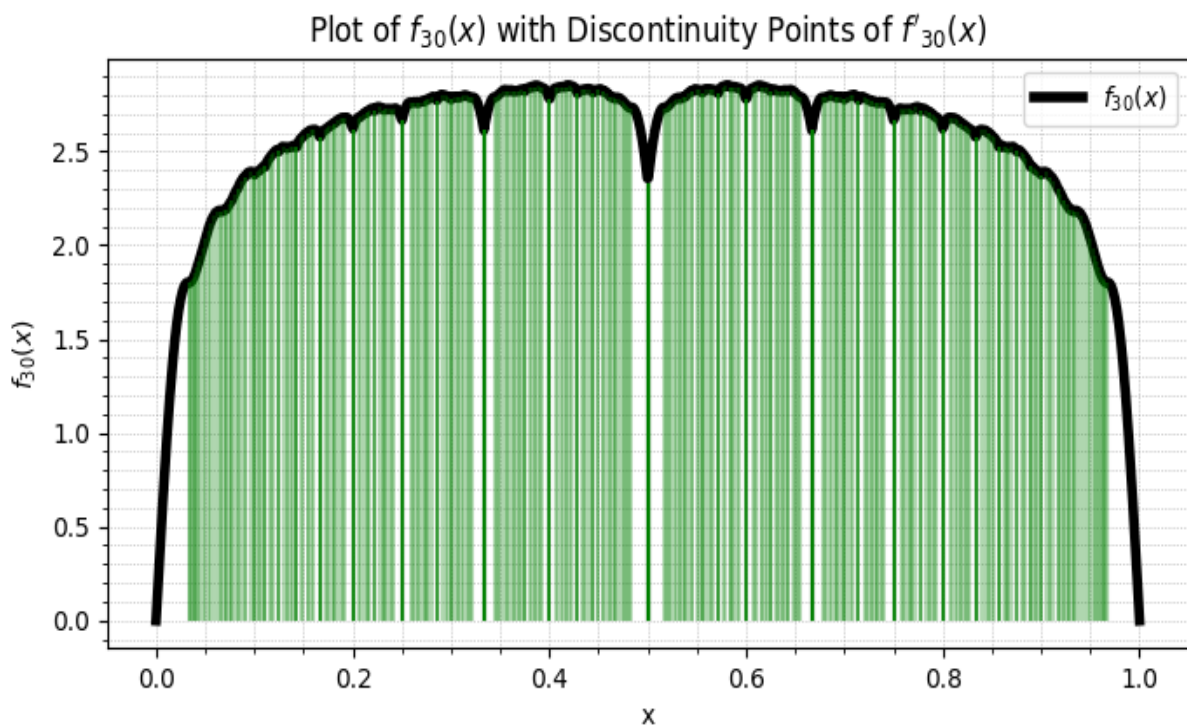


Figure 23: Plot of function $f_{30}(x)$ with vertical lines at discontinuity points.

10.2 Part (b)

I have implemented two methods: Trapezoidal integration and method of adaptive quadrature using `quad` from `scipy`. I would prefer the `quad` function for this integration task due to its high accuracy and adaptive nature, which effectively handles the discontinuities in the derivative of the integrand.

The analytical results (**shown together at the end**) confirms the accuracy of the numerical estimate obtained using `quad`. Other adaptive quadrature method such as Gauss Legendre quadrature can also be effective as they do not divide the whole domain into equally spaced points. Adaptive quadrature methods do importance sampling.

I have considered following points [2] for making my choice:

- The **quad** function uses an adaptive algorithm that dynamically subdivides the integration interval which is crucial for handling regions with sharp changes or discontinuities. And as shown in fig-23, there are lots of irregularities and change in values of the given function. So, this adaptive nature is quite beneficial.
- The adaptive algorithm increases the density of points in problematic regions, effectively dealing with discontinuities in the derivative.

Results:

- **Trapezoidal Rule:** ≈ 2.5309
- **quad (Adaptive Quadrature):** ≈ 2.5434
- **Gaussian Quadrature:** ≈ 2.5422
- **Analytical Result:** ≈ 2.5433

10.3 Part (c)

The integral to be calculated analytically is

$$I = \int_0^1 f_{30}(x) dx,$$

where function f_{30} is a summation series:

$$f_{30}(x) = \sum_{j=1}^{30} \frac{|\sin(j\pi x)|}{j}.$$

We can write the integral as follow now:

$$I = \int_0^1 \sum_{j=1}^{30} \frac{|\sin(j\pi x)|}{j} dx = \sum_{j=1}^{30} \int_0^1 \frac{|\sin(j\pi x)|}{j} dx.$$

Each term in the summation can be integrated separately:

$$\int_0^1 \frac{|\sin(j\pi x)|}{j} dx.$$

To integrate $\int_0^1 \frac{|\sin(j\pi x)|}{j} dx$, let us first consider the following integral:

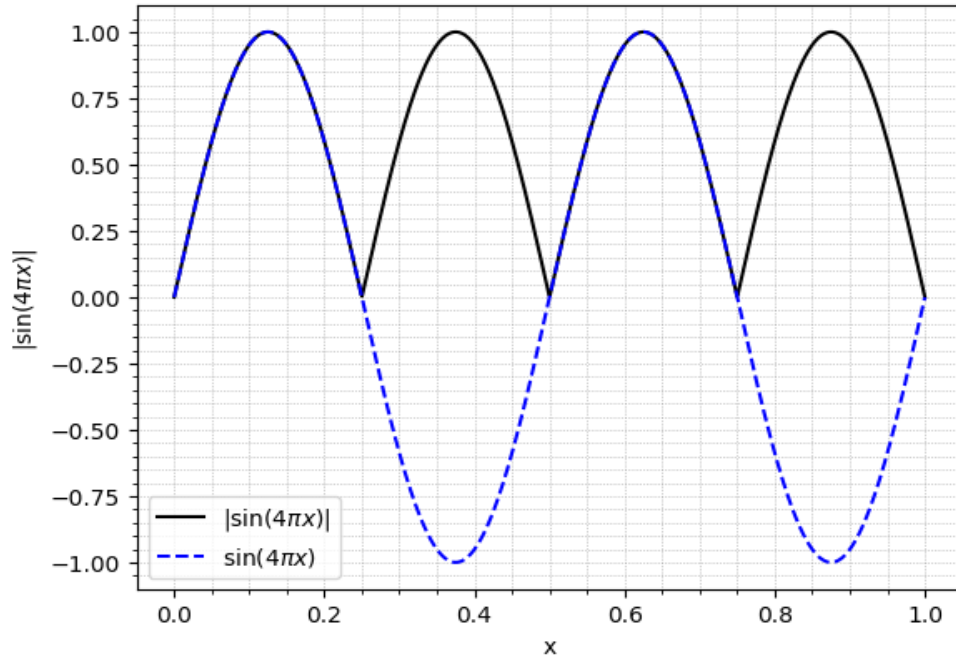
$$\int_0^1 |\sin(j\pi x)| dx.$$

For $j \geq 1$, the sine function completes half a period over the interval $[0, \frac{1}{j}]$ and then repeats itself. Thus, $|\sin(j\pi x)|$ is symmetric and periodic.

$$\implies \int_0^1 |\sin(j\pi x)| dx = 2 \int_0^{\frac{1}{2j}} \sin(j\pi x) dx.$$

Now, consider the case of $|\sin(4\pi x)|$. As shown in fig-24, the function $|\sin(4\pi x)|$ is symmetric over $[0, \frac{1}{2 \times 4}]$. From the figure, following information can also be concluded by observing the symmetry:

$$\int_0^{\frac{1}{2}} |\sin(4\pi x)| dx = 4 \int_0^{\frac{1}{2 \times 4}} \sin(4\pi x) dx.$$

Figure 24: Plot to demonstrate symmetry of $|\sin(4\pi x)|$

This observation can be extended for general case of j and we can write following:

$$\int_0^{\frac{1}{2}} |\sin(j\pi x)| dx = j \int_0^{\frac{1}{2j}} \sin(j\pi x) dx = j \times \left(\frac{1}{j\pi}\right) = \frac{1}{\pi}$$

$$\Rightarrow \int_0^1 |\sin(j\pi x)| dx = \frac{2}{\pi}.$$

So, each integral becomes:

$$\int_0^1 \frac{|\sin(j\pi x)|}{j} dx = \frac{2}{\pi j}.$$

Thus, the total integral I is:

$$I = \sum_{j=1}^{30} \int_0^1 \frac{|\sin(j\pi x)|}{j} dx = \sum_{j=1}^{30} \frac{2}{\pi j} = \frac{2}{\pi} \sum_{j=1}^{30} \frac{1}{j}.$$

Now, using Wolfram Alpha (also done using Python in my notebook),

$$\sum_{j=1}^{30} \frac{1}{j} \approx 3.9951$$

Thus, the integral I is approximately

$$\Rightarrow I \approx \frac{2}{\pi} \times 3.9951.$$

$$I \approx 2.5422.$$

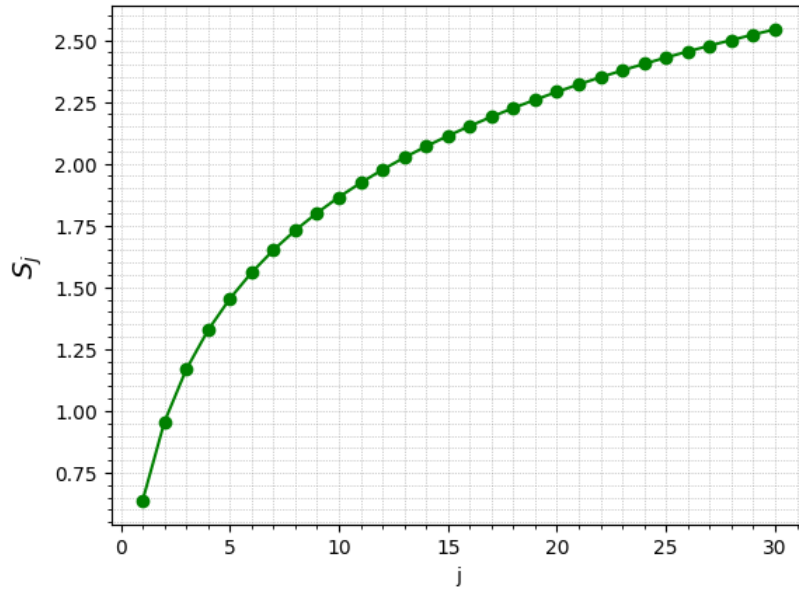


Figure 25: Integration result as each term is added after integrating.

The numerical result from the `quad` function (2.5434) closely matches the analytical result (2.5433), indicating that the `quad` function provides a very accurate approximation. The trapezoidal rule result (2.5309) is slightly less accurate but still reasonably close.

10.4 Appendix: Discontinuity of $f'_n(x)$ at $x = 0$

As x approaches 0 from the left,

$$\begin{aligned}
 \lim_{x \rightarrow 0^-} f'_n(x) &= \lim_{x \rightarrow 0-h} f'_n(x) \\
 &= \lim_{h \rightarrow 0} \sum_{j=1}^n \frac{\cos(-j\pi h) \cdot \pi \cdot \operatorname{sgn}(\sin(-j\pi h))}{j} \\
 &= \lim_{h \rightarrow 0} \sum_{j=1}^n \frac{\cos(j\pi h) \cdot \pi \cdot \operatorname{sgn}(-\sin(j\pi h))}{j} \\
 &= - \sum_{j=1}^n \frac{\pi}{j}
 \end{aligned}$$

As x approaches 0 from the right,

$$\begin{aligned}
 \lim_{x \rightarrow 0^+} f'_n(x) &= \lim_{x \rightarrow 0+h} f'_n(x) \\
 &= \lim_{h \rightarrow 0} \sum_{j=1}^n \frac{\cos(j\pi h) \cdot \pi \cdot \operatorname{sgn}(\sin(j\pi h))}{j} \\
 &= \lim_{h \rightarrow 0} \sum_{j=1}^n \frac{\cos(j\pi h) \cdot \pi \cdot \operatorname{sgn}(\sin(j\pi h))}{j} \\
 &= \sum_{j=1}^n \frac{\pi}{j}
 \end{aligned}$$

Since the left-hand limit and the right-hand limit of $f'_n(x)$ are not equal, the derivative is discontinuous at $x = 0$.

10.5 References

- [1] <https://www.derivative-calculator.net/>
- [2] https://en.wikipedia.org/wiki/Trapezoidal_rule
- [3] ACSI Lecture Notes, Dr. Huang

11 Parallel Programming[Dr. Wang]

11.1 Serial Code

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <chrono>
5  #include <iomanip>
6
7  using namespace std;
8
9  /**
10 * -----
11 * @brief function to generate prime numbers using the
12 *       Sieve of Eratosthenes algorithm.
13 *       https://www.topcoder.com/thrive/articles/sieve-of-eratosthenes-
14 *       algorithm#
15 * @param is_prime - array indicating whether each number is prime.
16 * @param limit    - Upper limit for generating primes.
17 * -----
18 */
19 void generate_primes(bool* is_prime, int limit) {
20
21     fill(is_prime, is_prime + limit + 1, true);
22     is_prime[0] = is_prime[1] = false;
23
24     for (int i = 2; i * i <= limit; ++i) {
25         if (is_prime[i]) {
26             for (int j = i * i; j <= limit; j += i) {
27                 is_prime[j] = false;
28             }
29         }
30     }
31 }
32
33
34 /**
35 * -----
36 * @brief Verifies Goldbach's conjecture for even numbers up to a given
37 *       limit.
38 *       Goldbach's conjecture states that every even integer greater than 2 can
39 *       be
40 *       expressed as the sum of two prime numbers.
41 * @param is_prime - Pointer to an array indicating whether each number is
42 *       prime.
43 * @param limit    - Upper limit for verifying Goldbach's conjecture.
44 * @return         - True if Goldbach's conjecture holds for all even
45 *       numbers
46 *                   up to the limit, false otherwise.
47 * -----

```

```
46  */
47  bool verify_goldbach(bool* is_prime, int limit) {
48      bool valid = true;
49
50      for (int n = 4; n <= limit; n += 2) {
51          bool found = false;
52          for (int i = 2; i <= n / 2; ++i) {
53              if (is_prime[i] && is_prime[n - i]) {
54                  found = true;
55                  break;
56              }
57          }
58          if (!found) {
59              cout << "Goldbach's conjecture is wrong for " << n << endl;
60              valid = false;
61          }
62      }
63      return valid;
64  }
65
66
67  int main() {
68
69      const int limit = 10000000;
70      bool* is_prime = new bool[limit + 1];
71
72      auto start = chrono::high_resolution_clock::now();
73      generate_primes(is_prime, limit);
74      auto end = chrono::high_resolution_clock::now();
75      cout << endl;
76
77      chrono::duration<double> diff = end - start;
78      cout << "Time to generate primes: " << diff.count() << " s" << endl;
79
80      start = chrono::high_resolution_clock::now();
81      bool result = verify_goldbach(is_prime, limit);
82      end = chrono::high_resolution_clock::now();
83      diff = end - start;
84
85      cout << "Time to verify Goldbach's conjecture: " << diff.count() << " s"
86      << endl;
87      cout << "Goldbach Conjecture: " << (result ? "Holds" : "False") << endl;
88      cout << endl;
89
90      delete[] is_prime;
91      return 0;
92  }
```

Output:

Time to generate primes: 0.0879415 s
Time to verify Goldbach's conjecture: 0.225632 s
Goldbach Conjecture: Holds

11.2 Parallel Code and Load balancing

The parallel code is implemented in OpenMP and for load balancing three different schedules with different chunk sizes have been tried. As shown in Table-12, the scalability is very good. The speedup is marginally better in case of Guided and Dynamic schedule in comparison to default static schedule.

```

1 bool verify_goldbach(bool* is_prime, int limit, omp_sched_t schedule_type,
2   int chunk_size) {
3
4   omp_set_schedule(schedule_type, chunk_size);
5
6   #pragma omp parallel for schedule(runtime)
7   for (int n = 4; n <= limit; n += 2) {
8       bool found = false;
9       for (int i = 2; i <= n / 2; ++i) {
10          if (is_prime[i] && is_prime[n - i]) {
11              found = true;
12              break;
13          }
14      }
15      if (!found) {
16          #pragma omp critical
17          {
18              cout << "Goldbach's conjecture is wrong for " << n << endl;
19              valid = false;
20          }
21      }
22  }
23  return valid;
24 }

```

The function `verify_goldbach` in my OpenMP code can be used with different schedule types. We can also change chunk size. A optimum chunk size can further help in reducing overhead and better load balancing.

Benefits of Adjusting Chunk Size ([ChatGpt](#)):

Load Balancing

- **Small Chunk Size:** Improves load balancing by assigning smaller portions to threads, preventing idle threads.
- **Large Chunk Size:** Reduces assignment overhead but can cause imbalance if iterations vary in execution time.

Reducing Overhead

- **Small Chunk Size:** Can increase overhead due to frequent task assignments.
- **Large Chunk Size:** Lowers task assignment overhead but may lead to idle threads with uneven workloads.

Threads	Time (s)	Goldbach Conjecture	Scalability
Static			
1	0.198131	Holds	1.00
2	0.108946	Holds	1.82
4	0.056187	Holds	3.53
8	0.0281674	Holds	7.03
16	0.0246524	Holds	8.04
28	0.0149682	Holds	13.23
Dynamic			
1	0.200183	Holds	1.00
2	0.110321	Holds	1.81
4	0.0564322	Holds	3.55
8	0.0281172	Holds	7.12
16	0.0165787	Holds	12.08
28	0.0148891	Holds	13.44
Guided			
1	0.198265	Holds	1.00
2	0.108231	Holds	1.83
4	0.0548089	Holds	3.62
8	0.027563	Holds	7.19
16	0.0167507	Holds	11.83
28	0.0146753	Holds	13.51

Table 12: Performance and Scalability of Different Schedules

11.3 Scalability of Code

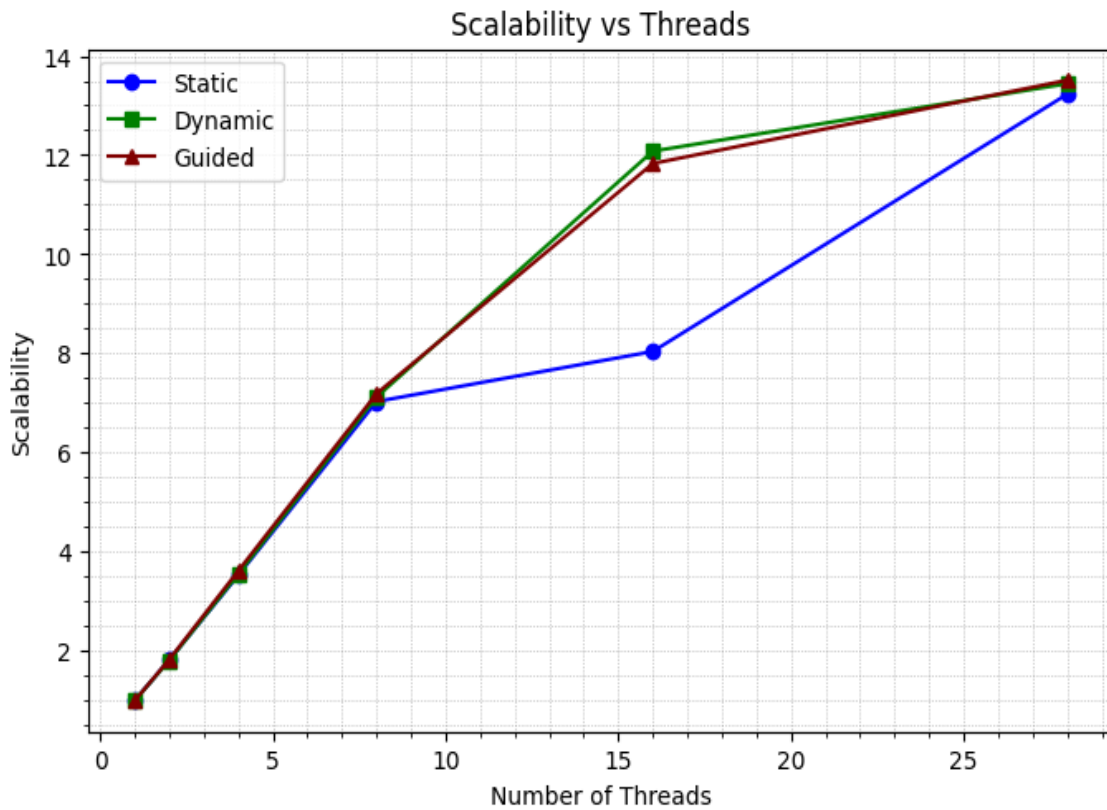


Figure 26: Scalability of code with different schedules.

NOTES:

- I have used classroom machines to run the OpenMP code. It has a maximum of 28 threads present.
- Compilation commands used:

```
-----  
[mt22t@class104 Q11-200973108]$ g++ -o openMP Q11_openMP.cpp -fopenmp  
[mt22t@class104 Q11-200973108]$ ./openMP  
-----
```

11.4 References

1. <https://www.topcoder.com/thrive/articles/sieve-of-eratosthenes-algorithm#>
2. <https://www.openmp.org/wp-content/uploads/SC19-FlorinaCiorba-Dynamic.pdf>
3. HPC Lecture Notes, Dr. Wang

References

- [1] Wikipedia contributors. Regularized least squares — wikipedia, the free encyclopedia, 2024. Accessed: 2024-05-21.
- [2] Víctor Elvira, Luca Martino, and Pau Closas. Importance gaussian quadrature. *IEEE Transactions on Signal Processing*, 69:474–488, 2020.
- [3] GeeksforGeeks. Particle swarm optimization (pso) – an overview, Year not specified.
- [4] Uwe Meyer-Baese and U Meyer-Baese. *Digital signal processing with field programmable gate arrays*, volume 65. Springer, 2007.
- [5] Michael A. Saunders. Least squares and eigenvalues. <http://www.cs.cornell.edu/courses/cs6241/2019sp/readings/Saunders-least-squares-CME338.pdf>, 2019.
- [6] Wikipedia contributors. Metropolis–hastings algorithm. https://en.wikipedia.org/wiki/Metropolis%E2%80%93Hastings_algorithm.
- [7] Wikipedia contributors. Particle swarm optimization. https://en.wikipedia.org/wiki/Particle_swarm_optimization.
- [8] Allen Woolf. Using eigenvalues and eigenvectors to find stability and solve odes, 2024. Accessed on 2024-05-25.